

Московский авиационный институт (национальный исследовательский
университет)

Компьютерные науки и прикладная математика

Прикладная математика и информатика

Бакалавриат

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

**Мобильное приложение для отслеживания и построения гоночных
маршрутов парусного спорта**

Студент: Гиголаев Антон Александрович

Группа: М8О-409Б-22

Руководитель: Дзюба Дмитрий Владимирович
Старший преподаватель

Москва — 2026

СПИСОК ИСПОЛНИТЕЛЕЙ

Гиголаев Антон Александрович — Аналитическая часть, проектирование, реализация и описание мобильного приложения RegattaTracker.

РЕФЕРАТ

Выпускная квалификационная работа состоит из 84 страниц, 5 рисунков, 8 таблиц, 39 использованных источников и 1 приложения.

РЕГАТА, МОБИЛЬНОЕ ПРИЛОЖЕНИЕ, GPS-ТРЕКИНГ, IMU, КОМПЛЕМЕНТАРНЫЙ ФИЛЬТР, OFFLINE-FIRST, ЛОКАЛЬНОЕ ХРАНИЛИЩЕ, СИНХРОНИЗАЦИЯ ДАННЫХ, ГОНОЧНЫЙ КОМПЬЮТЕР, ЭКСПОРТ ТЕЛЕМЕТРИИ

В работе разработано мобильное приложение RegattaTracker для сопровождения парусных регат. Приложение собирает GPS- и IMU-данные на смартфоне, сохраняет их локально, продолжает работу при потере связи и передает накопленную телеметрию после восстановления канала.

Цель работы — разработать мобильный клиент, пригодный для регаты в условиях нестабильной сети и ограниченного заряда устройства. Для этого проанализированы существующие решения, выбран стек Flutter с native bridge и локальным хранилищем SQLite + Drift, спроектированы локальная модель данных и pipeline телеметрии, реализованы сбор сенсоров, синхронизация, export и race computer.

Результатом стал мобильный клиент с локальной БД, очередью повторной отправки, обработкой GPS- и IMU-потокa, экспортом артефактов гонки и средствами диагностики. Контрольные прогоны показали расход заряда 4-7% за 30--60 минут при целевых частотах GPS около 1 Гц и IMU около 50 Гц.

СОДЕРЖАНИЕ

СПИСОК ИСПОЛНИТЕЛЕЙ	1
РЕФЕРАТ	2
ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ	6
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	7
ВВЕДЕНИЕ	8
1 Аналитическая часть	10
1.1 Предметная область и постановка практической задачи . . .	10
1.2 Актуальность разработки	10
1.3 Особенности предметной области, влияющие на архитектуру приложения	11
1.4 Анализ рынка и существующих решений	12
1.5 Выводы по аналитической части	16
2 Обоснование выбора технологий, языков и методов	18
2.1 Критерии выбора технологического стека	18
2.2 Обоснование выбора кроссплатформенного подхода	18
2.3 Обоснование выбора языков программирования	21
2.4 Обоснование выбора архитектурного стиля	22
2.5 Обоснование выбора локального хранилища: SQLite + Drift	23
2.6 Обоснование выбора методов сбора сенсорных данных . . .	27
2.7 Обоснование выбора метода фильтрации и вычисления ориентации	31
2.8 Обоснование выбора модели сетевого взаимодействия и синхронизации	34
2.9 Обоснование выбора картографической технологии	35
2.10 Обоснование выбора форматов экспорта	36
2.11 Обоснование выбора методов тестирования и оценки энергопотребления	37
2.12 Выводы по главе	38
3 Проектирование мобильного приложения RegattaTracker	40
3.1 Цель и задачи проектирования	40
3.2 Функциональные требования к мобильному приложению .	40

3.3	Нефункциональные требования	42
3.4	Положение мобильного приложения в общей системе	42
3.5	Проектирование внутренней архитектуры мобильного приложения	44
3.6	Проектирование локальной модели данных	46
3.7	Проектирование потока сбора, обработки и синхронизации телеметрии	47
3.8	Проектирование алгоритмического блока ``гоночный компьютер"	50
3.9	Проектирование пользовательских сценариев и интерфейса	52
3.10	Выводы по главе	53
4	Реализация мобильного приложения и проверка проектных решений	54
4.1	Общая структура проекта	54
4.2	Реализация композиции зависимостей и модульной сборки .	54
4.3	Реализация локального хранилища как рабочего центра приложения	54
4.4	Реализация механизма tracking-session	55
4.5	Реализация native bridge и платформенного сбора данных . .	56
4.6	Реализация pipeline приема и обработки sample	57
4.7	Реализация live-передачи и устойчивой синхронизации . . .	58
4.8	Реализация sensor fusion и вычисления derived metrics	59
4.9	Реализация подсистемы ``гоночный компьютер"	60
4.10	Реализация экспортной подсистемы	61
4.11	Реализация пользовательских сценариев и связь с интерфейсом	62
4.12	Реализация тестирования	63
4.13	Оценка энергопотребления и устойчивости работы	64
4.14	Соответствие результата исходным требованиям	66
4.15	Ограничения текущей реализации и направления развития .	68
4.16	Выводы по главе	69
	ЗАКЛЮЧЕНИЕ	70
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	71

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящей выпускной квалификационной работе применяют следующие термины с соответствующими определениями:

Гоночный компьютер — программный модуль мобильного приложения, формирующий прикладные навигационные и тактические показатели на основе локальной телеметрии, состояния дистанции и судейских событий

Комплементарный фильтр — алгоритм оценки ориентации, объединяющий данные акселерометра, гироскопа и магнитометра с различными частотными характеристиками

Локальная сессия трекинга — интервал времени, в течение которого мобильное приложение ведет сбор, сохранение и последующую синхронизацию телеметрии конкретного пользователя

Offline-first архитектура — архитектурный подход, при котором приложение сохраняет работоспособность и целостность данных при отсутствии сетевого соединения, используя локальное хранилище как основной источник рабочего состояния

Предобработка телеметрии — совокупность операций по фильтрации, структурированию, нормализации и преобразованию сырых сенсорных данных в прикладные метрики

Синхронизация телеметрии — процесс передачи локально накопленных данных и связанных с ними событий на внешний сервер после восстановления сетевой доступности

Sensor fusion — алгоритмическое объединение данных нескольких сенсоров для получения более устойчивых и информативных оценок состояния объекта

Телеметрия регаты — совокупность пространственных, временных, инерциальных и событийных данных, характеризующих ход парусной гонки

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящей выпускной квалификационной работе применяют следующие сокращения и обозначения:

API — Application Programming Interface, интерфейс программирования приложений

CSV — Comma-Separated Values, текстовый формат табличных данных

DTO — Data Transfer Object, объект передачи данных

EKF — Extended Kalman Filter, расширенный фильтр Калмана

GNSS — Global Navigation Satellite System, глобальная навигационная спутниковая система

GPX — GPS Exchange Format, формат обмена GPS-треками

GPS — Global Positioning System, глобальная система позиционирования

HTTP — HyperText Transfer Protocol, протокол передачи гипертекста

IMU — Inertial Measurement Unit, инерциальный измерительный модуль

JSON — JavaScript Object Notation, текстовый формат представления структурированных данных

NMEA — National Marine Electronics Association, семейство протоколов обмена морскими навигационными данными

OSM — OpenStreetMap, открытая картографическая платформа

REST — Representational State Transfer, архитектурный стиль сетевого взаимодействия

SQL — Structured Query Language, язык структурированных запросов

UI — User Interface, пользовательский интерфейс

WAL — Write-Ahead Logging, режим журналирования SQLite

ВВЕДЕНИЕ

Во время парусной регаты важны не только итоговые результаты, но и телеметрия, которая появляется прямо на воде: траектория старта, маневры, изменение курса, положение относительно линии старта и дистанции, крен и скорость поворота. Обычные спортивные трекеры обычно ограничиваются записью GPS-маршрута, а специализированные приборы заметно повышают стоимость оснащения. В проекте RegattaTracker смартфон используется как основной полевой модуль, который собирает и сохраняет данные гонки в ходе заезда.

Проблема проекта состоит в том, что приложение должно непрерывно работать в фоне, собирать GPS- и IMU-данные, не терять их при пропадании связи, сохранять локально и дозагружать позже, не разряжая устройство за одну гонку. Без этого участник и судья получают либо неполную картину гонки, либо инструмент, которым невозможно пользоваться на протяжении всей дистанции.

Объектом исследования является мобильное приложение RegattaTracker, предназначенное для сопровождения парусных регат в ролях участника и судьи. Предметом исследования являются архитектурные и алгоритмические решения, которые обеспечивают сбор GPS- и IMU-данных, их локальную предобработку, хранение, синхронизацию и использование внутри мобильного клиента.

Цель работы — разработать мобильное приложение, обеспечивающее надежный сбор и предобработку данных сенсоров смартфона для сопровождения парусной регаты в условиях нестабильной связи и ограниченного заряда устройства.

Для достижения цели в работе решаются следующие задачи:

1. выполнить анализ предметной области и существующих программных решений, применяемых для сопровождения регат, навигации и спортивного трекинга;
2. выбрать технологический стек, архитектурный подход, методы хранения данных и алгоритм оценки ориентации;
3. спроектировать внутреннюю архитектуру мобильного приложения, локальную модель данных и поток обработки телеметрии;

4. реализовать мобильный клиент с поддержкой ролей участника и судьи, локальной БД, очереди синхронизации и экспортных механизмов;
5. реализовать подсистему сбора и предобработки GPS- и IMU-данных, включая native bridge и комплементарный фильтр;
6. проверить соответствие результата исходным требованиям и проанализировать показатели энергопотребления и устойчивости работы.

В работе использованы сравнительный анализ существующих решений, методы проектирования программной архитектуры, мобильной разработки, обработки сигналов и тестирования программного обеспечения.

Практическая ценность работы состоит в том, что разработанное приложение можно использовать при клубных и учебно-тренировочных регатах, где важны фоновый сбор телеметрии, работа без сети и последующий разбор трека без обязательного специализированного оборудования.

1 АНАЛИТИЧЕСКАЯ ЧАСТЬ

1.1 Предметная область и постановка практической задачи

В парусной регате результат зависит не только от скорости яхты, но и от того, насколько точно экипаж работает со стартом, дистанцией, ветром и маневрами. Траектория движения здесь сама по себе является частью спортивного результата: по ней видно, как судно заходило на линию, когда меняло курс и насколько стабильно проходило дистанцию.

Поэтому для анализа гонки недостаточно редкой записи GPS-точек. Нужны непрерывный сбор телеметрии, привязка к контексту гонки и вычисление прикладных метрик: скорости, курса, угловой скорости поворота, крена, положения относительно стартовой линии и судейских событий.

Практическая проблема проекта RegattaTracker состоит в том, что массовые трекары и навигационные приложения обычно не рассчитаны на сценарий регаты. Они либо записывают только базовый GPS-трек, либо ориентированы на круизную навигацию, либо требуют отдельного оборудования. В результате клубным и учебно-тренировочным регатам не хватает доступного инструмента, который одновременно:

- работает на обычном смартфоне;
- обеспечивает устойчивый сбор GPS- и IMU-данных;
- функционирует в условиях нестабильной связи;
- поддерживает роли участника и судьи;
- предоставляет данные как для оперативного использования на воде, так и для последующего анализа.

Задача проекта формулируется как разработка энергоэффективного мобильного модуля для iOS и Android, способного в фоновом режиме собирать данные GPS, акселерометра, гироскопа и магнитометра, выполнять первичную фильтрацию, вычислять производные параметры движения и сохранять их локально с последующей синхронизацией и экспортом.

1.2 Актуальность разработки

Клубным и учебно-тренировочным регатам нужен мобильный инструмент, который можно запустить на обычном смартфоне без отдельного трекара и без постоянной зависимости от сети. На практике доступные решения

закрывают только часть задачи: одни лучше подходят для replay, другие — для судейской организации, третьи — для работы с фирменным оборудованием и подписочной моделью [1]–[11].

Сложность проекта связана не с отображением карты, а с фоновым сбором GPS- и IMU-данных, локальным хранением, повторной синхронизацией и контролем энергопотребления под ограничениями Android и iOS [12], [13], [14], [15]. Поэтому RegattaTracker разрабатывается не как очередной GPS-трекер, а как мобильный сенсорный модуль для сопровождения регаты.

1.3 Особенности предметной области, влияющие на архитектуру приложения

Архитектуру приложения определяют несколько особенностей парусной регаты.

1.3.1 Изменчивость среды

Судно движется в среде, где в течение одной гонки могут заметно меняться ветер, волна и течение. Это означает, что телеметрия должна иметь не только пространственную, но и временную точность. Значение, записанное раз в 10–15 секунд, уже может быть недостаточным для анализа старта, маневра огибания или изменения курса на лавировке.

1.3.2 Важность стартовой процедуры

Старт в парусной гонке часто определяет весь дальнейший сценарий соревнования. Поэтому приложению недостаточно просто показывать карту. Оно должно поддерживать стартовый отсчет, контроль положения относительно линии старта, оценку времени до сигнала и, при наличии соответствующей логики, выдачу рекомендаций в предстартовый период.

1.3.3 Нестабильность связи

Даже на акваториях вблизи берега возможны кратковременные потери мобильной связи, а на крупных водоемах или в неблагоприятных погодных условиях такие потери становятся закономерными. Следовательно, архитектура мобильного приложения не может строиться по принципу ``получил точку и сразу передал на сервер''. Требуется локальное персистентное хранилище и повторяемый механизм догрузки данных после восстановления связи.

1.3.4 Ограниченность энергии

Сбор геопозиции и инерциальных данных, работа в фоне, периодическая запись на диск и сетевой обмен создают постоянную нагрузку на батарею устройства. Если архитектура будет спроектирована без учета энергопотребления, приложение либо слишком быстро разрядит устройство, либо будет остановлено системой по причинам, связанным с ограничениями фона и энергосбережения. Поэтому тема энергоэффективности является не дополнительной, а базовой.

1.3.5 Двухролевая модель использования

В проекте RegattaTracker предусмотрены как минимум две прикладные роли:

- участник;
- судья.

Участнику важны оперативные тактические подсказки и надежная запись трека. Судье важны постановка дистанции, запуск стартовой процедуры, фиксация судейских событий и контроль хода гонки. Это требует не просто двух экранов, а двух различающихся пользовательских сценариев поверх общей технологической базы.

1.3.6 Высокая ценность пост-анализа

Даже если во время гонки мобильное приложение используется как ``гоночный компьютер'', ключевая ценность телеметрии часто раскрывается позже. После завершения заезда тренер, судья или экипаж могут анализировать replay, сопоставлять траектории, крены, скорость, участки потерь и преимущества на дистанции. Следовательно, экспорт и структурированное сохранение данных являются обязательной частью системы, а не второстепенной возможностью.

1.4 Анализ рынка и существующих решений

Анализ рынка необходим для ответа на два вопроса:

1. Какие функции в рассматриваемой предметной области уже считаются базовыми?
2. Какие ограничения и незакрытые потребности оставляют пространство для разработки собственного решения?

Для удобства рассмотрения решения можно разделить на две группы:

- прямые конкуренты, ориентированные на проведение и сопровождение регат;
- косвенные конкуренты, решающие смежные задачи трекинга, навигации, спортивной аналитики и отображения морских данных.

1.4.1 Прямые конкуренты

Regatta Hero

Regatta Hero автоматизирует клубные регаты: поддерживает стартовую последовательность, GPS-трекинг, автоматическую фиксацию финиша, scoring и replay через браузер [1], [2]. Это сильный ориентир по судейскому сценарию, но продукт меньше фокусируется на локальной IMU-обработке и offline-first сборе телеметрии на смартфоне.

Sailmon App

Sailmon строит экосистему вокруг приложения, облачной платформы и фирменного оборудования MAX/MAX Mini, сочетая replay, трекинг и пост-анализ [3], [4]. Сильная сторона решения — цельный пользовательский контур, но зависимость от внешних устройств повышает порог входа по сравнению со сценарием ``только смартфон".

TackTracker

TackTracker развивает live tracking и replay с облачным сервером, поддержкой отдельных трекеров и loggers, а также публикацией результатов гонки [5]. Решение хорошо закрывает инфраструктуру соревнования, но смартфон не рассматривается в нем как основной сенсорный узел.

PredictWind Race Tracker

PredictWind Race Tracker предлагает GPS-трекинг гонок без отдельного оборудования, live-наблюдение и replay для участников и зрителей [6]. Это удобный способ быстро запустить трекинг, но судейская логика и гоночный компьютер здесь выражены заметно слабее.

RaceQs

RaceQs известен прежде всего как сервис 3D replay и аналитики гоночных треков [7], [8]. Для послегоночного разбора это сильный пример, но не полноценный мобильный клиент для ведения регаты на воде.

1.4.2 Косвенные конкуренты

Waterspeed

Waterspeed показывает, как можно сделать удобный consumer-продукт для водных активностей на обычном смартфоне: скорость, дистанция, маршрут, live sharing и интеграции с wearables [9], [10]. При этом в нем нет судейского сценария и логики парусной регаты.

iRegatta

iRegatta — тактическое приложение для iPhone со стартовым таймером, расчетом линии старта, полярными диаграммами и поддержкой AIS/NMEA [11]. По духу это близкий мобильный гоночный инструмент, но он больше рассчитан на личное использование, чем на связанный контур ``участник — судья — сервер — replay".

SailGrib WR

SailGrib WR делает акцент на погоде, маршрутизации, картах и интеграции AIS/NMEA, а регатные функции в нем вторичны [16]. Это полезный навигационный инструмент, но не система сопровождения клубной гонки.

Navionics Boating

Navionics Boating предоставляет офлайн-карты, планирование маршрутов, GPX-экспорт и погодные слои [17], [18]. Для проекта это ориентир по картографическому UX, но не конкурент по регатной логике и работе с телеметрией гонки.

Vakaros

Vakaros — аппаратно-программная экосистема с устройством Atlas 2, двухдиапазонным GNSS 25 Гц и подписочной логикой RaceSense для стартовой линии и OCS-сценариев [19]–[21]. По точности это высокий ориентир, но высокая стоимость и аппаратная зависимость оставляют заметную нишу для решения на обычном смартфоне.

1.4.3 Сравнительная таблица решений

В таблице 1.1 приведено сопоставление прямых и косвенных конкурентных решений с точки зрения функций, которые покрывает разрабатываемый мобильный клиент.

Таблица 1.1 – Сравнительная характеристика существующих решений рынка

Решение	Основной фокус	Сильные стороны	Ограничения относительно RegattaTracker
Regatta Hero	Клубное проведение регат	Стартовая логика, scoring, replay, веб-наблюдение [1], [2]	Менее выраженный акцент на локальной IMU-обработке и offline-first мобильном сенсорном модуле
Sailmon	Экосистема трекинга, replay и hardware	Сильная аналитика, событийная логика, интеграция с устройствами [3], [4]	Зависимость от фирменного оборудования, высокий порог входа
TackTracker	Live tracking и replay	Богатый функционал воспроизведения, облачное хранение [5]	Опора на отдельные трекары и инфраструктуру соревнования
PredictWind Race Tracker	GPS-трекинг гонок	Простота запуска, репутация в морской навигации [6]	Гоночный компьютер и судейский режим выражены слабее
RaceQs	Replay и аналитика	Сильный post-race сценарий [7], [8]	Не является полноценным мобильным регатным клиентом
Waterspeed	Спортивный watersports tracking	Понятный пользовательский опыт, speed analytics [9], [10]	Нет судейской модели и регатной логики

Таблица 1.1 – Продолжение таблицы 1.1

Решение	Основной фокус	Сильные стороны	Ограничения относительно RegattaTracker
iRegatta	Тактическое мобильное приложение	Стартовый таймер, линия старта, AIS/NMEA [11]	Ориентация на индивидуальное использование, а не на связанную экосистему
SailGrib WR	Навигация и routing	Погода, маршрутизация, карты, AIS/NMEA [16]	Не предназначен для сопровождения регаты как события
Navionics Boating	Морские карты и навигация	Офлайн-карты, маршруты, GPX, погодные слои [17], [18]	Не решает задачу спортивного трекинга гонки
Vakaros	Специализированное оборудование	Очень высокая точность, RaceSense, 25 Гц GNSS [19]–[21]	Высокая стоимость и аппаратная зависимость

1.4.4 Выводы по результатам анализа рынка

Сравнение существующих продуктов показывает, что рынок уже сформирован, но решения закрывают только отдельные части сценария: судейскую организацию, replay, навигацию или работу с фирменным оборудованием. Ниша RegattaTracker определяется более узко и конкретно: приложение должно работать на обычном смартфоне, сохранять телеметрию без сети, поддерживать роли участника и судьи и использовать сервер в первую очередь для синхронизации и последующего анализа.

1.5 Выводы по аналитической части

Анализ предметной области и рынка показывает следующее.

1. Парусная регата предъявляет к мобильному приложению более жест-

кие требования, чем типовой GPS-трекинг активности, так как здесь важны не только координаты, но и контекст гонки, стартовая логика, точность временной привязки и устойчивость при работе в фоне.

2. Основная практическая проблема состоит в том, что клубным и учебно-тренировочным регатам нужен доступный инструмент, который собирает телеметрию на обычном смартфоне и не перестает работать при потере связи.
3. Существующие решения подтверждают рыночную востребованность подобного класса систем, но не закрывают полностью нишу кроссплатформенного offline-first мобильного клиента, сочетающего роли участника и судьи.
4. Наиболее значимыми инженерными требованиями к разрабатываемому приложению становятся:
 - энергоэффективный фоновый сбор GPS и IMU;
 - локальное надежное хранение данных;
 - повторяемая синхронизация после восстановления связи;
 - вычисление полезных производных метрик на клиенте;
 - поддержка функций гоночного компьютера;
 - экспорт и последующий анализ записанного трека.

Эти выводы непосредственно переходят в следующую главу, посвященную выбору технологий и методов реализации.

2 ОБОСНОВАНИЕ ВЫБОРА ТЕХНОЛОГИЙ, ЯЗЫКОВ И МЕТОДОВ

2.1 Критерии выбора технологического стека

Стек для RegattaTracker подбирался под задачу фонового сенсорного клиента, а не по принципу популярности технологий. В качестве основных критериев были выделены:

- поддержка Android и iOS;
- возможность работы с нативными API сенсоров и геолокации;
- достаточная производительность при сборе и обработке телеметрии;
- возможность построения единой кодовой базы для бизнес-логики и интерфейса;
- поддержка локального хранилища и фоновых сценариев;
- управляемая сложность проекта и сопровождения;
- наличие развитой документации, инструментов тестирования и сообщества.

Стек должен был сократить дублирование кода и при этом не закрыть доступ к нативным API, локальному хранению и фоновым сценариям.

2.2 Обоснование выбора кроссплатформенного подхода

Для реализации клиента рассматривались три стратегических пути:

- разработка двух нативных приложений: Kotlin/Java для Android и Swift для iOS;
- разработка кроссплатформенного приложения на Flutter с нативными расширениями;
- разработка кроссплатформенного приложения на React Native или аналогичном фреймворке.

2.2.1 Сравнение с полностью нативной реализацией

Полностью нативная разработка дает прямой доступ к системным API фона, геолокации и сенсоров. Для трекинга на мобильной платформе это сильный аргумент.

Однако полностью нативный подход имеет и значимые недостатки:

- необходимо поддерживать две существенно разные кодовые базы;
- дублируется практически вся прикладная логика: модели данных, син-

- хронизация, экспорт, бизнес-правила, значительная часть тестов;
- возрастает стоимость изменений и исправлений;
- усложняется сопровождение и доказательство архитектурной целостности проекта.

В этом проекте нативный путь означал бы две кодовые базы и двойную поддержку моделей данных, синхронизации, экспорта и тестов. Для одного мобильного клиента это слишком дорогой вариант.

2.2.2 Сравнение Flutter и React Native

React Native был естественной альтернативой Flutter: экосистема зрелая, а разработчиков на рынке много. Но для проекта с фоновым трекингом и обработкой сенсорных данных важнее были предсказуемое исполнение, явная граница между кроссплатформенной и нативной логикой и простой способ упаковать платформенный код в собственный плагин.

Flutter позиционируется как фреймворк для создания нативно компилируемых многоплатформенных приложений из единой кодовой базы [22], [23]. Если стандартных пакетов недостаточно, платформенную часть можно вынести в собственные плагины и platform channels [24], [25]. Для проекта это означало следующее:

- интерфейс и бизнес-логика пишутся единообразно на Dart;
- доступ к Android- и iOS-специфике можно вынести в отдельный plugin-пакет;
- граница между кроссплатформенной и нативной логикой остается явной и контролируемой.

Эта схема хорошо соответствует RegattaTracker: общая прикладная логика остается единой, а чувствительные к платформе механизмы сбора сенсоров и фоновой работы изолируются в native bridge.

2.2.3 Почему выбран Flutter с нативным bridge

Flutter с нативными плагинами был выбран по трем причинам.

Во-первых, основная логика проекта не зависит от конкретной ОС: роли участника и судьи, работа с локальной БД, экспорт, синхронизация, race computer и UI удобнее развивать в одной кодовой базе.

Во-вторых, различия между платформами вынесены в отдельный слой. В проекте это сделано через пакет `regatta_sensor_bridge`, внутри которого находятся:

- Dart-контракт и модель обмена;
- Kotlin-реализация Android-сервиса;
- Swift-реализация iOS-плагина.

В-третьих, такой подход уменьшает дублирование кода, но не заставляет отказываться от нативных возможностей. Для RegattaTracker это не компромисс ради скорости разработки, а способ совместить единый клиент и глубокий доступ к платформе.

2.2.4 Ограничения выбранного решения

Для объективности необходимо зафиксировать и ограничения выбранного стека.

1. Работа в фоне и доступ к сенсорам на мобильных платформах все равно зависят от нативного слоя. Следовательно, кроссплатформенность не устраняет необходимость глубокой платформенной проработки.
2. Зрелость платформенных реализаций различается: Android bridge реализован существенно глубже, тогда как iOS-компонент на текущем этапе охватывает каналы взаимодействия, управление состоянием сессии и обработку разрешений, но еще не доведен до того же уровня фонового GPS/IMU-сбора.
3. При любом кроссплатформенном подходе необходима аккуратная сериализация сообщений между Dart и host-кодом, иначе возрастает риск ошибок при передаче типов и статусов.

Для текущей постановки задачи эти ограничения приемлемы.

2.2.5 Сравнительная таблица вариантов реализации

В таблице 2.1 отражены ключевые различия между альтернативными вариантами реализации мобильного клиента.

Таблица 2.1 – Сравнение стратегий реализации мобильного клиента

Критерий	Нативные приложения	Flutter + native plugins	React Native + native modules
Доступ к платформенным API	Максимальный	Высокий, через plugins/channels	Высокий, но требует более сложной мостовой интеграции

Таблица 2.1 – Продолжение таблицы 2.1

Критерий	Нативные приложения	Flutter + native plugins	React Native + native modules
Единая кодовая база UI и бизнес-логики	Нет	Да	Да
Скорость разработки двух платформ	Ниже	Выше	Выше
Стоимость сопровождения	Высокая	Средняя	Средняя
Предсказуемость для сенсорного приложения	Высокая	Высокая при правильном native bridge	Зависит от JS-bridge и выбранных модулей
Уместность при ограниченных ресурсах	Низкая	Высокая	Средняя

Таблица показывает, что нативный подход выигрывает только в абсолютной свободе работы с платформой, но проигрывает в стоимости разработки и сопровождения. React Native оставался рабочим вариантом, однако Flutter лучше подошел для разделения общей логики и чувствительного платформенного кода.

2.3 Обоснование выбора языков программирования

В проекте используются три языка: Dart для общей прикладной логики, Kotlin для Android-части native bridge и Swift для iOS-плагинов.

2.3.1 Dart

Dart выбран как основной язык проекта, потому что на нем реализованы не только интерфейс, но и прикладное ядро: локальная модель данных, синхронизация, экспорт, `TrackingSessionService`, `TrackingSampleIngestionService`, `RaceComputerService`, `ComplementaryFilter` и связанные use case. Для RegattaTracker это важно, потому что большая часть доменной логики должна одинаково

работать на Android и iOS.

2.3.2 Kotlin

Kotlin используется в Android-части native bridge, потому что именно здесь нужны прямой доступ к `FusedLocationProviderClient`, `SensorManager`, foreground service, уведомлениям и системе разрешений [12], [13], [24], [26]. Эти механизмы завязаны на Android SDK и не могут быть корректно реализованы только на Dart.

2.3.3 Swift

Swift применен для iOS-плагины, потому что он дает доступ к `Core Location` и `Core Motion` [14], [15], [27]. Текущая iOS-реализация проще Android-части, но распределение языков остается логичным: Dart закрывает общую прикладную систему, Kotlin — Android-специфику, Swift — iOS-специфику.

2.4 Обоснование выбора архитектурного стиля

По результатам анализа кодовой базы мобильное приложение RegattaTracker построено как feature-oriented layered application. Это означает, что функциональность организована по предметным модулям, а внутри модулей прослеживается разделение на presentation, application, domain и data/local storage.

Такой подход выбран не случайно.

2.4.1 Почему не monolithic UI-first структура

Для небольшого учебного приложения допустима структура, где вся логика группируется вокруг экранов. Однако для RegattaTracker это было бы ошибкой. Проект содержит:

- несколько ролей пользователя;
- локальную БД;
- синхронизацию;
- сенсорный pipeline;
- экспорт;
- алгоритмические расчеты;
- нативный plugin.

Если бы эти части были связаны напрямую через экраны и виджеты, приложение быстро стало бы трудным для сопровождения и тестирования.

2.4.2 Почему layered + feature-oriented подход подходит лучше

Разделение на функциональные области и слои дало несколько преимуществ.

1. Локализация ответственности. Например, `sensor fusion`, `race computer`, `export` и `tracking` не смешиваются в одном файле или одном `controller`.
2. Удобство тестирования. Наличие отдельных сервисов и `use case` позволяет тестировать поведение вне UI, что подтверждается существующими `unit`-тестами проекта [28].
3. Масштабируемость. Добавление новых форматов экспорта, новых расчетных метрик или новых сценариев судьи не требует переписывать архитектуру целиком.
4. Ясная интеграция с `dependency injection`. В проекте центральной точкой сборки зависимостей выступает `app_dependencies.dart`, что делает архитектурный граф явным.

2.4.3 Почему для проекта важен принцип offline-first

В контексте `RegattaTracker` `offline-first` является не модным архитектурным лозунгом, а прямым следствием условий эксплуатации. Если приложение должно собирать телеметрию на акватории, оно не может считать сервер единственным источником истины в ходе гонки.

Выбор локальной БД в качестве операционного ядра приводит к важному свойству: приложение сначала фиксирует состояние и данные у себя, а затем синхронизирует их наружу. Эта схема:

- снижает риск потери данных при обрыве сети;
- упрощает восстановление после сбоев;
- позволяет использовать локальные данные для `race computer` и экспорта;
- дает воспроизводимый журнал синхронизации.

С инженерной точки зрения это сильное решение, потому что оно согласуется и с реальностью мобильной среды, и с предметной областью.

2.5 Обоснование выбора локального хранилища: SQLite + Drift

2.5.1 Возможные альтернативы

Для локального хранилища мобильного приложения можно было рассматривать несколько вариантов:

- файлы JSON/CSV;
- SharedPreferences/Key-Value storage;
- SQLite с ручным SQL;
- ORM/DAO-надстройки над SQLite;
- объектные БД, например Realm или ObjectBox.

2.5.2 Почему не файлы и не key-value storage

Файловое хранение хорошо подходит для простых логов и экспорта, но плохо подходит для сложных выборок, индексации, конкурентного доступа и транзакционности. В проекте необходимо:

- связывать GPS-точки с сессиями;
- хранить chunks IMU;
- хранить derived metrics;
- поддерживать sync queue;
- отслеживать export jobs;
- быстро извлекать последние точки, агрегаты и диагностические данные.

Такая задача уже по своей природе реляционна. Key-value storage тем более не подходит: он удобен для настроек и токенов, но не для телеметрического журнала.

2.5.3 Почему не чистый SQLite без типобезопасной надстройки

Чистый SQLite остается мощным вариантом, однако в проекте среднего размера ручное сопровождение SQL, схем, маппинга строк в объекты и миграций увеличивает трудоемкость и вероятность ошибок. Особенно это касается проектов, где база активно используется в нескольких подсистемах.

Drift позиционируется как reactive persistence library для Dart и Flutter, дающая типобезопасность, stream queries, генерацию кода и инструменты миграций [29], [30]. Для RegattaTracker это означает, что можно использовать преимущества SQLite, не теряя при этом удобства работы из Dart-кода.

2.5.4 Почему выбрана именно связка SQLite + Drift

Данный выбор оправдан следующими причинами.

1. SQLite встраивается прямо в мобильное приложение, не требует отдельного серверного процесса и хорошо подходит для локального хранения телеметрии.
2. Drift сохраняет преимущества SQLite, но добавляет:

- типобезопасную модель таблиц;
 - генерацию DAO-уровня;
 - удобные выборки;
 - реактивные stream-обновления;
 - поддержку миграций.
3. В проекте уже реально используются несколько связанных таблиц и фоновые операции, что делает полноценную БД предпочтительнее файловой схемы.
 4. Использование SQLite не закрывает путь к эффективной работе с бинарными полезными нагрузками: IMU-данные могут храниться как BLOB-поля, что особенно важно для производительности и компактности.

2.5.5 Почему выбрано хранение IMU в виде бинарных блоков

Один из важнейших архитектурных выборов проекта касается формы хранения IMU-данных. В текущей реализации они не записываются построчно в виде ``одна строка на один сенсорный sample'', а группируются в бинарные chunks.

Это решение оправдано сразу по нескольким причинам.

Во-первых, частота IMU в проекте составляет 50 Гц. Даже при умеренной продолжительности сессии количество отдельных sample-объектов быстро становится очень большим. Построчная запись каждого sample в реляционную таблицу приведет к:

- росту объема индексов;
- увеличению числа операций вставки;
- повышению нагрузки на GC и сериализацию;
- замедлению выборок и экспорта.

Во-вторых, для многих вычислительных сценариев IMU удобнее обрабатывать пачками, а не поштучно. Chunk-ориентированная схема лучше соответствует pipeline ``записать - импортировать - обработать в isolate - получить derived metrics''.

В-третьих, BLOB-подход хорошо сочетается с SQLite и позволяет сохранить атомарность хранения, не создавая внешнюю файловую зависимость для каждой микропорции данных.

2.5.6 Обоснование использования WAL-режима

В коде мобильной БД явно включен режим `PRAGMA journal_mode = WAL`, а также дополнительные параметры `synchronous = NORMAL`, `temp_store = MEMORY` и увеличенный `cache size`. Это не случайная оптимизация, а продуманное решение.

Согласно документации SQLite, Write-Ahead Logging позволяет отделить путь записи от пути чтения, благодаря чему чтения могут продолжаться параллельно с записью, а `commit` выполняется посредством добавления записи в WAL-журнал [31]. Для проекта RegattaTracker это дает несколько преимуществ:

- ускоряется запись телеметрии;
- повышается устойчивость при одновременной записи и чтении локальных данных;
- уменьшается вероятность того, что UI или экспорт будут блокировать поступление новых `sample`.

Поэтому связка SQLite + Drift + WAL является не просто ``популярной'', а технически адекватной задаче интенсивной фоновой телеметрии на мобильном устройстве.

2.5.7 Сравнение вариантов локального хранения

Сравнение вариантов локального хранения данных представлено в таблице 2.2.

Таблица 2.2 – Сравнение вариантов локального хранения телеметрии

Подход	Преимущества	Недостатки	Вывод для RegattaTracker
Файлы JSON/CSV	Простота, легкий экспорт	Нет индексации, сложные выборки, слабая транзакционность	Подходит только как экспорт, но не как рабочее хранилище
SharedPreferences / key-value	Минимальная сложность	Не подходит для временных рядов и связанных сущностей	Годится только для настроек

Таблица 2.2 – Продолжение таблицы 2.2

Подход	Преимущества	Недостатки	Вывод для RegattaTracker
Realm / ObjectBox	Удобная объектная модель, хорошие скорости	Дополнительная экосистема, меньшая прозрачность SQL-уровня, менее естественная BLOB-схема под аналитические выборки	Возможный вариант, но менее естественный для исследовательского offline-first pipeline
SQLite + ручной SQL	Полный контроль, зрелость технологии	Высокая трудоемкость поддержки и миграций	Возможен, но избыточно дорог в сопровождении
SQLite + Drift	Зрелая БД, типобезопасность, миграции, удобные выборки, WAL, BLOB	Требуется генерация кода и дисциплины схем	Наиболее рациональный вариант для текущего проекта

2.6 Обоснование выбора методов сбора сенсорных данных

2.6.1 Почему в проекте недостаточно было готовых универсальных Flutter-пакетов

На первый взгляд задача доступа к геолокации и датчикам в Flutter может решаться набором готовых community-пакетов. Такой путь действительно подходит для многих стандартных приложений. Однако для RegattaTracker он оказался недостаточным по следующим причинам:

- необходимо управлять состоянием долгоживущей tracking-session, а не просто подписаться на поток координат;
- необходимо одновременно обрабатывать GPS и IMU;
- необходимо учитывать профиль трекинга в зависимости от стадии гон-

ки;

- необходимо обеспечить стабильную работу в фоне;
- необходимо передавать в Dart не только отдельные точки, но и диагностическое состояние, статистику потерь sample и пути к временным пакетам данных;
- необходимо явно контролировать runtime permissions и платформенные ограничения.

Именно поэтому в проекте был выбран путь собственного bridge-пакета вместо механического использования нескольких готовых зависимостей поверх стандартных абстракций.

2.6.2 Почему для Android выбран FusedLocationProvider

В Android-части проекта координаты собираются через Google Play services location stack. Это решение соответствует официальным рекомендациям Android по построению энергосбалансированного геотрекинга и выбору сценария request-updates в зависимости от потребности приложения [12], [32], [33].

Использование FusedLocationProvider дает несколько преимуществ по сравнению с прямой работой только через LocationManager:

- система сама комбинирует источники позиционирования;
- можно точнее управлять приоритетом точности и интервалами;
- уменьшается вероятность дублирования низкоуровневой логики, уже реализованной в Google Play services;
- проще поддерживать предсказуемое поведение на широком спектре устройств Android.

В текущей реализации Android-сервиса используется `Priority.PRIORITY_HIGH_ACCURACY`, а минимальный интервал обновления не опускается ниже одной секунды. Это полностью согласуется с целевой конфигурацией GPS-сбора не ниже 1 Гц.

2.6.3 Почему для IMU на Android выбран SensorManager

Для акселерометра, гироскопа и магнитометра в Android используется `SensorManager` и прямая регистрация слушателей. Это естественный выбор, поскольку именно системный API Android предоставляет доступ к стандартным motion и position sensors [26].

Такой подход позволяет:

- выбирать требуемую частоту опроса;
- независимо управлять тремя источниками данных;
- получать временные метки и сырые значения без лишних промежуточных абстракций;
- синхронизировать поведение сервиса с жизненным циклом tracking-session.

В проекте частота IMU задается через конфигурацию сессии и в текущей реализации устанавливается в 50 Гц, что соответствует целевой конфигурации сенсорного модуля.

2.6.4 Почему выбран foreground service для Android

Одной из ключевых проблем мобильного трекинга является работа в фоне. Начиная с современных версий Android, ОС существенно ограничивает доступ к геоданным и долгоживущим фоновым задачам для приложений, не использующих корректную foreground-модель [13].

В проекте выбран foreground service с постоянным уведомлением, потому что только такой путь позволяет реализовать честный и устойчивый сценарий фонового сбора телеметрии при длительной гонке. Код Android-плагины подтверждает:

- создание notification channel;
- запуск `startForeground(...)`;
- маркировку сервиса как location-foreground-service;
- отдельную обработку разрешений `ACCESS_FINE_LOCATION`, `ACCESS_BACKGROUND_LOCATION`, `ACTIVITY_RECOGNITION`, `POST_NOTIFICATIONS`.

С точки зрения архитектуры это важный момент. Проект не пытается обойти ограничения ОС сомнительными обходными путями, а реализует сбор данных в легитимной системной модели.

2.6.5 Профили трекинга как средство баланса точности и энергопотребления

Особого внимания заслуживает то, что приложение не использует единственный ``жестко прибитый" режим GPS-сбора. В коде Android bridge присутствует понятие tracking profile, для которого задаются различные политики интервала и минимальной дистанции обновления:

- `prestartPrecision;`
- `raceCruise;`
- `markRoundingPrecision;`
- `paused.`

Такое решение отражает грамотную инженерную идею: на разных этапах гонки цена ошибки различна. В предстартовой фазе и при огибании знака важна максимальная точность и минимальная фильтрация по дистанции. На ровном гоночном участке допустимо сохранить частоту 1 Гц, но ввести порог минимального перемещения, чтобы снизить объем лишних обновлений и сетевого трафика.

Следовательно, выбор профилей трекинга является одним из ключевых механизмов адаптации системы к противоречивым требованиям точности и автономности.

2.6.6 Почему iOS рассматривается как обязательная платформа, но с оговоркой по зрелости

С точки зрения постановки задачи мобильный модуль должен быть пригоден для работы на iOS и Android. Документация Apple подтверждает наличие необходимых платформенных возможностей:

- Core Location поддерживает фоновые обновления местоположения и модель явного разрешения на постоянный доступ [14], [15];
- Core Motion предоставляет работу с motion-данными и акселерометрией [27].

Текущая iOS-реализация охватывает каналы взаимодействия, управление `tracking-session`, запросы разрешений и `health payload`, однако пока не повторяет глубину Android-сервиса в части `long-running GPS/IMU`-сбора и фоновое накопление пакетов телеметрии.

Выбранный технологический подход поддерживает обе платформы, но в текущей проектной реализации Android выступает как более зрелая целевая платформа для сенсорного модуля.

2.7 Обоснование выбора метода фильтрации и вычисления ориентации

2.7.1 Постановка задачи фильтрации

Сырые данные акселерометра, гироскопа и магнитометра полезны сами по себе лишь ограниченно. Для практического применения необходимо вычислять производные характеристики: крен, наклон, курс, угловую скорость изменения курса и косвенные признаки устойчивости движения.

Проблема состоит в том, что каждый сенсор обладает собственными недостатками:

- акселерометр чувствителен к линейным ускорениям и вибрациям;
- гироскоп хорошо отражает краткосрочную динамику, но подвержен дрейфу при интегрировании;
- магнитометр позволяет оценивать направление относительно магнитного поля Земли, но чувствителен к помехам и локальным искажениям.

Следовательно, для получения полезной оценки ориентации необходимо объединять данные нескольких сенсоров, то есть применять sensor fusion.

2.7.2 Рассматриваемые альтернативы

В рамках данной работы логично рассматривать три основных класса методов:

- complementary filter;
- Kalman filter / extended Kalman filter;
- специализированные ориентационные фильтры семейства Mahony/Madgwick.

2.7.3 Complementary filter

Смысл complementary filter заключается в том, что высокочастотная составляющая сигнала берется из гироскопа, а низкочастотная опорная компонента - из акселерометра и магнитометра. В простейшей форме это можно записать как:

$$\theta_k = \alpha(\theta_{k-1} + \omega_k \Delta t) + (1 - \alpha)\theta_k^{ref}$$

где:

- θ_k — текущая оценка угла;
- ω_k — угловая скорость гироскопа;

- Δt — шаг дискретизации;
- θ_k^{ref} — опорная оценка, вычисленная по акселерометру/магнитометру;
- α — коэффициент доверия к интегрированной гироскопической составляющей.

Подобные подходы лежат в основе complementary attitude filters, описанных, в частности, в работах Mahony и соавторов [34]. Их сильная сторона — вычислительная простота, хорошая интерпретируемость и устойчивость для embedded/mobile-сценариев.

2.7.4 Kalman filter

Kalman-подход дает более строгую вероятностную модель оценки состояния системы и позволяет учитывать шумы и ошибки измерения через ковариационные матрицы. Классическое введение в этот класс методов можно найти у Welch и Bishop [35].

Преимущества Kalman-подхода:

- строгая математическая постановка;
- возможность учета сложной модели ошибок;
- высокая гибкость для многосенсорного слияния;
- хорошие результаты при корректной настройке.

Недостатки для рассматриваемого проекта:

- более высокая вычислительная и концептуальная сложность;
- необходимость аккуратного подбора модели процесса и параметров шумов;
- возрастание сложности тестирования и сопровождения;
- избыточность для первого этапа мобильного сенсорного модуля, где важны устойчивость, предсказуемость и скорость реализации.

2.7.5 Фильтры Mahony и Madgwick

Фильтры Mahony и Madgwick часто рассматриваются как практические промежуточные решения между очень простыми complementary-схемами и более тяжелыми EKF-подходами. В частности, алгоритм Madgwick разрабатывался как эффективный и вычислительно недорогой ориентировочный фильтр для IMU/MARG-систем [36].

Эти подходы представляют собой перспективное направление развития проекта. Однако их внедрение требует более тщательной калибровки и верификации на реальных полевых данных.

2.7.6 Почему в проекте выбран complementary filter

Complementary filter выбран в текущей реализации RegattaTracker по следующим причинам.

1. Complementary filter обеспечивает устойчивую оценку ориентации при умеренной вычислительной стоимости и хорошо подходит для непрерывной работы на обычном смартфоне.
2. Частота IMU составляет 50 Гц, а вычисления должны выполняться стабильно без заметного влияния на UX и батарею.
3. В проекте дополнительно реализованы калибровка и последующая генерация derived metrics, что усиливает практическую полезность даже относительно простой fusion-схемы.
4. Архитектура не блокирует будущий переход к более сложным методам. Если в дальнейшем потребуется повысить точность на основе расширенной экспериментальной базы, complementary filter может быть заменен на Mahony, Madgwick или EKF без кардинального пересмотра всей архитектуры хранения и синхронизации.

Следовательно, выбор complementary filter является не компромиссом ``из-за упрощения'', а рациональным инженерным решением для первой зрелой версии мобильного модуля.

2.7.7 Дополнительные замечания о вычислении курса и крена

Для парусного приложения особенно важны не только формальные углы ориентации, но и их прикладная интерпретация. Например:

- крен помогает судить о загрузке парусов, влиянии порывов ветра и качестве балансировки;
- курс и heading помогают анализировать удержание линии и тактические маневры;
- turn rate важен для оценки интенсивности перекладки и входа в поворот;
- сглаженное ускорение помогает отделять устойчивое движение от кратковременных вибраций и ударов корпуса о волну.

Поэтому алгоритм фильтрации в проекте следует рассматривать не изолированно, а как часть цепочки ``сырые сенсоры -> устойчивая оценка ориентации -> прикладные спортивные метрики''. Это усиливает обоснование выбранного метода: даже если более сложный фильтр потенциально мог бы

дать прирост точности, практическая ценность определяется тем, насколько уверенно и стабильно можно получить именно прикладные выходные параметры на мобильном устройстве.

2.8 Обоснование выбора модели сетевого взаимодействия и синхронизации

2.8.1 Почему backend не является центром логики мобильного клиента

Хотя мобильное приложение взаимодействует с backend-инфраструктурой, сама архитектура мобильной части не строится как тонкий клиент. Это принципиально важно. Клиент не ожидает от сервера постоянного подтверждения каждого своего действия, а способен:

- создавать локальную tracking-session;
- продолжать сбор данных без сети;
- копить queue на отправку;
- использовать локальные вычисленные метрики;
- выполнять экспорт на устройстве.

Подобный подход защищает приложение от сетевой хрупкости и соответствует offline-first логике.

2.8.2 Почему выбрана схема immediate upload + persistent retry

В проекте используется комбинированная модель:

- при появлении GPS-точки выполняется попытка немедленной live-передачи;
- при неудаче создается запись в sync_queue;
- отдельный worker позднее повторяет отправку.

Это решение лучше, чем крайние альтернативы.

Если бы приложение только отправляло live-данные без локальной очереди, то при сетевом сбое возникала бы прямая потеря телеметрии. Если бы приложение всегда сначала только копило локальные данные и ничего не отправляло в ходе гонки, терялась бы возможность live-наблюдения и серверного сопровождения соревнования.

Комбинированный подход позволяет совместить обе цели:

- обеспечить live-сценарий при наличии сети;
- не терять данные при отсутствии сети.

2.8.3 Почему для обмена выбраны HTTP/JSON и REST-подобные endpoint'ы

С точки зрения мобильного клиента было возможно рассматривать и более ``реалтаймовые" схемы, включая WebSocket или MQTT. Однако для первой зрелой версии проекта HTTP/JSON имеет ряд преимуществ:

- простота интеграции и диагностики;
- понятность контрактов;
- удобство ретраев;
- хорошая совместимость с существующей backend-структурой;
- отсутствие необходимости поддерживать постоянное соединение на слабой сети.

Для мобильного приложения в условиях нестабильной связи повторяемый запрос к endpoint'у зачастую оказывается надежнее, чем поддержка долгоживущего realtime-channel. С учетом того, что наиболее критичным является не мгновенность в миллисекундах, а гарантированность записи спортивно значимых данных, такой выбор оправдан.

2.9 Обоснование выбора картографической технологии

В проектном контексте упоминается использование OpenStreetMap как картографической основы. Это соответствует архитектуре решения, где требуется:

- интерактивное отображение положения судна;
- отображение дистанции и контрольных объектов;
- независимость от дорогостоящих закрытых картографических SDK;
- возможность разработки и тестирования без жесткой привязки к коммерческой подписке.

OpenStreetMap представляет собой открытый картографический проект с развитой экосистемой данных и тайлов [37]. Для Flutter-клиента удобным выбором является пакет `flutter_map`, который широко используется для интеграции карт OSM в Flutter-приложениях [38].

Почему это решение рационально:

1. Оно снижает инфраструктурную зависимость на этапе разработки и пилотной эксплуатации.
2. Оно хорошо подходит для инженерных и учебно-исследовательских

проектов.

3. Оно обеспечивает достаточный уровень визуализации для трекинга и отладки.

При этом необходимо понимать, что для промышленной эксплуатации на больших соревнованиях может потребоваться отдельная проработка тайл-сервиса, кеширования и лицензионных вопросов, однако для задач рассматриваемого проекта выбранный путь является оправданным.

2.10 Обоснование выбора форматов экспорта

Требования к мобильному модулю прямо указывают на необходимость поддержки популярных форматов экспорта данных. В проекте фактически реализованы:

- CSV;
- GPX;
- GeoJSON;
- diagnostics JSON;
- ZIP bundle.

Этот набор можно признать удачным по следующим причинам.

2.10.1 CSV

CSV удобен для табличного анализа, загрузки в офисные приложения и быстрой ручной обработки. Это полезный формат для преподавателя, исследователя, тренера и любого пользователя, которому требуется быстрый доступ к рядам значений без специализированного GIS-инструмента.

2.10.2 GPX

GPX является де-факто стандартом для обмена GPS-треками. Поддержка GPX повышает совместимость проекта с навигационными и картографическими приложениями и делает экспорт осмысленным вне экосистемы самого проекта.

2.10.3 GeoJSON

GeoJSON удобен для картографических веб-систем и последующей интеграции с GIS-инструментами. Он особенно полезен, если трек необходимо визуализировать в веб-контексте или обрабатывать в скриптах и аналитических пайплайнах.

2.10.4 Diagnostics JSON

С инженерной точки зрения наличие отдельного диагностического экспорта является сильной особенностью проекта. Он позволяет фиксировать не только сами координаты, но и условия качества данных:

- среднюю частоту GPS;
- среднюю частоту IMU;
- наличие потерь;
- состояние разрешений;
- лаг синхронизации;
- признаки влияния на батарею;
- состояние фоновых сервисов.

Это важный аргумент зрелости проекта: система ориентирована не только на пользовательский функционал, но и на наблюдаемость собственного поведения.

2.10.5 ZIP bundle

ZIP-пакет объединяет несколько экспортов в один переносимый артефакт. Это удобно как для архивирования результатов гонки, так и для передачи набора данных преподавателю, тренеру или внешнему аналитическому сервису.

2.11 Обоснование выбора методов тестирования и оценки энергопотребления

2.11.1 Почему тестирование должно быть многоуровневым

Для такого проекта, как RegattaTracker, недостаточно только ручной проверки экранов. Приложение содержит:

- бизнес-логику сессий;
- sensor fusion;
- локальное хранилище;
- экспорт;
- синхронизацию;
- судейский сценарий.

Следовательно, нужны как минимум:

- unit-тесты прикладной логики;
- интеграционные проверки взаимодействия модулей;

- widget/UI-проверки;
- сценарные испытания на устройстве.

Flutter официально поддерживает такие уровни тестирования [28].

2.11.2 Почему контроль энергопотребления должен опираться на профилировщики платформы

Требование по энергоэффективности нельзя закрыть субъективной формулировкой ``приложение работает нормально''. Для Android необходимо использовать системные инструменты профилирования и анализа потребления батареи, включая Android Studio Profiler и связанный с ним Power Profiler / Battery tooling [39].

Причина проста: влияние на батарею складывается из нескольких источников:

- GPS;
- IMU;
- фоновый сервис;
- записи в БД;
- сетевого обмена;
- обновлений UI.

Следовательно, методика оценки должна фиксировать:

- длительность теста;
- режим трекинга;
- сеть включена/отключена;
- экран включен/выключен;
- изменение заряда;
- количество собранных GPS и IMU sample;
- наличие потерь и повторных отправок.

Такая методика будет описана далее в разделе, посвященном реализации и проверке.

2.12 Выводы по главе

Выбор технологий в RegattaTracker опирается на требования самой задачи: единая кроссплатформенная логика реализована на Flutter, доступ к критичным платформенным возможностям вынесен в native bridge, локальное хранение построено на SQLite + Drift, а для sensor fusion выбран компле-

ментарный фильтр. Этот стек закрывает фоновый сбор телеметрии, offline-first сценарий и последующую обработку данных без лишнего дублирования кода.

3 ПРОЕКТИРОВАНИЕ МОБИЛЬНОГО ПРИЛОЖЕНИЯ REGATTATRACKER

3.1 Цель и задачи проектирования

Цель проектирования состояла в разработке архитектуры мобильного приложения, которое:

- обеспечивает сбор GPS и IMU-данных;
- работает в фоне;
- не теряет данные при отсутствии сети;
- поддерживает роли участника и судьи;
- предоставляет оперативную и постфактум полезную информацию;
- может быть расширено без архитектурного слома.

Из этой цели следуют основные задачи проектирования:

1. определить функциональные и нефункциональные требования;
2. задать место мобильного клиента в общей системе;
3. спроектировать внутреннюю модульную архитектуру;
4. определить локальную модель данных;
5. описать pipeline сбора, предобработки и синхронизации данных;
6. предусмотреть способы тестирования, экспорта и последующего развития.

3.2 Функциональные требования к мобильному приложению

На основе исходных требований проекта и анализа репозитория мобильного приложения можно выделить следующие функциональные требования.

3.2.1 Требования к роли участника

Приложение в режиме участника должно:

- проходить аутентификацию в системе;
- присоединяться к гонке или активной сессии;
- запускать и останавливать трекинг;
- собирать GPS-координаты не реже 1 Гц;
- собирать IMU-данные с частотой 50 Гц;
- отображать ключевые тактические показатели;
- выполнять локальную запись и последующую синхронизацию;

- предоставлять данные для последующего экспорта.

3.2.2 Требования к роли судьи

Приложение в режиме судьи должно:

- создавать или выбирать регату;
- задавать конфигурацию дистанции;
- запускать стартовую процедуру;
- фиксировать события гонки;
- управлять состоянием гоночной сессии;
- передавать судейские действия в связанную экосистему.

3.2.3 Требования к аналитическим и сервисным функциям

Приложение должно:

- вычислять крен, курс и производные метрики;
- хранить контекст гонки локально;
- поддерживать экспорт в популярных форматах;
- формировать диагностический снимок состояния системы;
- поддерживать повторную синхронизацию после потери сети.

3.2.4 Матрица пользовательских сценариев

Матрица ключевых пользовательских сценариев приведена в таблице

3.1.

Таблица 3.1 – Матрица пользовательских сценариев мобильного приложения

Сценарий	Участник	Судья	Обязательность
Вход в систему	Да	Да	Обязательно
Выбор активной регаты	Да	Да	Обязательно
Запуск трекинга	Да	Да	Обязательно
Просмотр карты и текущего положения	Да	Да	Обязательно
Получение стартового отсчета	Да	Да	Обязательно
Контроль линии старта	Да	Да	Обязательно
Постановка дистанции	Нет	Да	Обязательно для судьи
Фиксация судейского события	Нет	Да	Обязательно для судьи
Просмотр derived metrics	Да	Частично	Желательно
Экспорт данных	Да	Да	Желательно

Эта матрица показывает, что приложение проектируется не как две отдельные программы, а как единая система с общим технологическим ядром и различающимися прикладными сценариями поверх него.

3.3 Нефункциональные требования

3.3.1 Надежность

Приложение не должно терять уже собранную телеметрию при кратковременном отсутствии связи, перезапуске UI или временной недоступности backend-сервисов.

3.3.2 Производительность

Архитектура должна выдерживать непрерывный поток GPS- и IMU-данных без заметной деградации пользовательского интерфейса. Для этого тяжелые вычисления должны быть отделены от слоя отображения.

3.3.3 Энергоэффективность

Система должна работать в фоне без чрезмерного разряда батареи, используя адаптивные профили трекинга и избегая избыточных сетевых операций.

3.3.4 Масштабируемость

Добавление новых производных метрик, форматов экспорта, сценариев анализа и даже новых алгоритмов fusion не должно требовать переписывания всей архитектуры.

3.3.5 Тестопригодность

Ключевая логика должна быть выделена в сервисы и use cases, пригодные для unit-тестирования без обязательного запуска UI.

3.4 Положение мобильного приложения в общей системе

Мобильное приложение работает не изолированно, а внутри связанной системы, в которую входят:

- мобильных пользователей в ролях участника и судьи;
- backend-сервисы аутентификации, управления гонкой и приема телеметрии;
- хранилища серверной части;
- веб-компоненты последующего отображения и аналитики.

Мобильное приложение не сводится к роли экрана для backend'a. Оно

содержит локальное хранилище, собственные алгоритмы и логику принятия решений в пределах мобильной сессии.

3.4.1 Контекстная диаграмма системы

Функциональное окружение мобильного приложения RegattaTracker показано на рисунке 3.1. Диаграмма фиксирует, что центральным элементом системы выступает мобильный клиент, тогда как backend-сервисы и веб-портал выступают внешними компонентами программного комплекса.

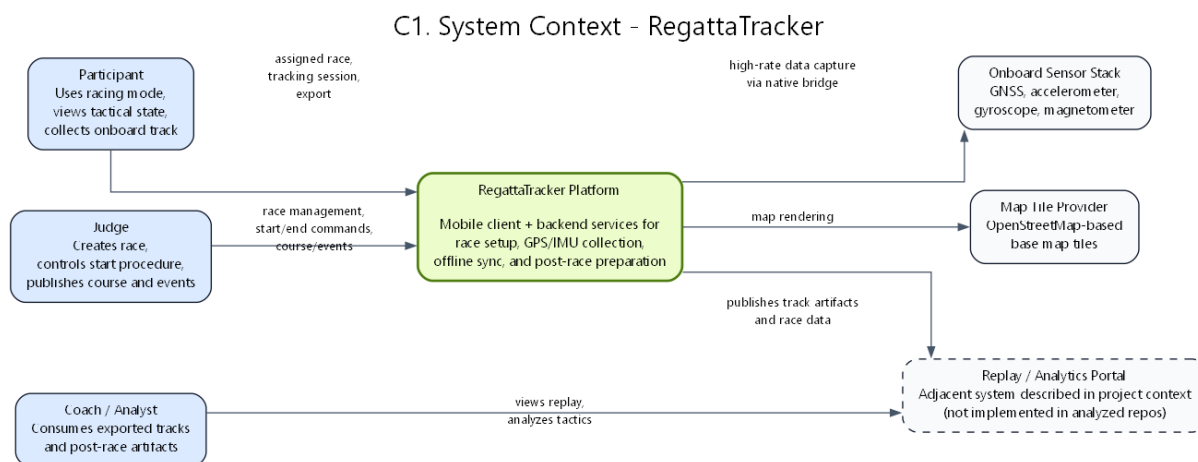


Рисунок 3.1 – Контекстная диаграмма системы RegattaTracker

Как следует из рисунка 3.1, участник и судья взаимодействуют с одним мобильным приложением, однако используют его в разных прикладных ролях. Смежный серверный контур обеспечивает аутентификацию, прием телеметрии и координацию гонки, а аналитический веб-контур потребляет уже подготовленные артефакты. Такое представление удерживает фокус анализа на мобильной части, не разрывая ее связь с остальной системой.

3.4.2 Контейнерная диаграмма взаимодействия с backend-инфраструктурой

Внешние интеграции мобильного приложения с серверной частью программного комплекса показаны на рисунке 3.2. Данная диаграмма используется в работе не для детального анализа backend-реализации, а для фиксации тех контейнеров, с которыми мобильный клиент действительно взаимодействует в ходе гонки.

На рисунке 3.2 видно, что клиент обращается к серверу через API-шлюз, использует сервисы аутентификации, управления гонками и приема телеметрии, но при этом сохраняет собственную локальную БД, native bridge

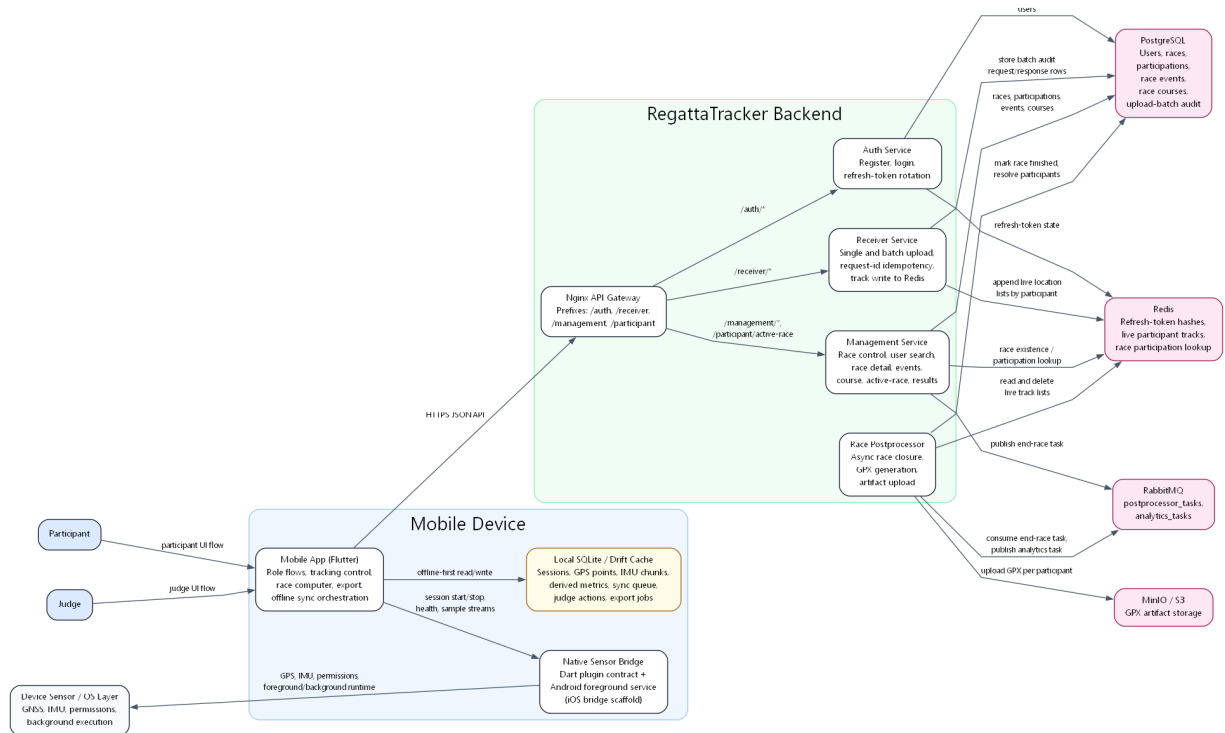


Рисунок 3.2 – Контейнерная архитектура программного комплекса RegattaTracker

и алгоритмические блоки. Сервер здесь рассматривается как внешняя интеграционная среда; более подробная ER-диаграмма серверной модели вынесена в приложение А.

3.5 Проектирование внутренней архитектуры мобильного приложения

По результатам анализа исходного кода мобильный клиент можно представить как набор связанных подсистем:

- presentation;
- authentication/session;
- tracking;
- sensor bridge;
- local storage;
- sync;
- race computer;
- export;
- judge flow.

3.5.1 Компонентная архитектура мобильного приложения

Внутренняя структура мобильного клиента показана на рисунке 3.3. Эта схема отражает декомпозицию приложения на прикладные подсистемы и подтверждает, что бизнес-логика не растворена в экранном коде.

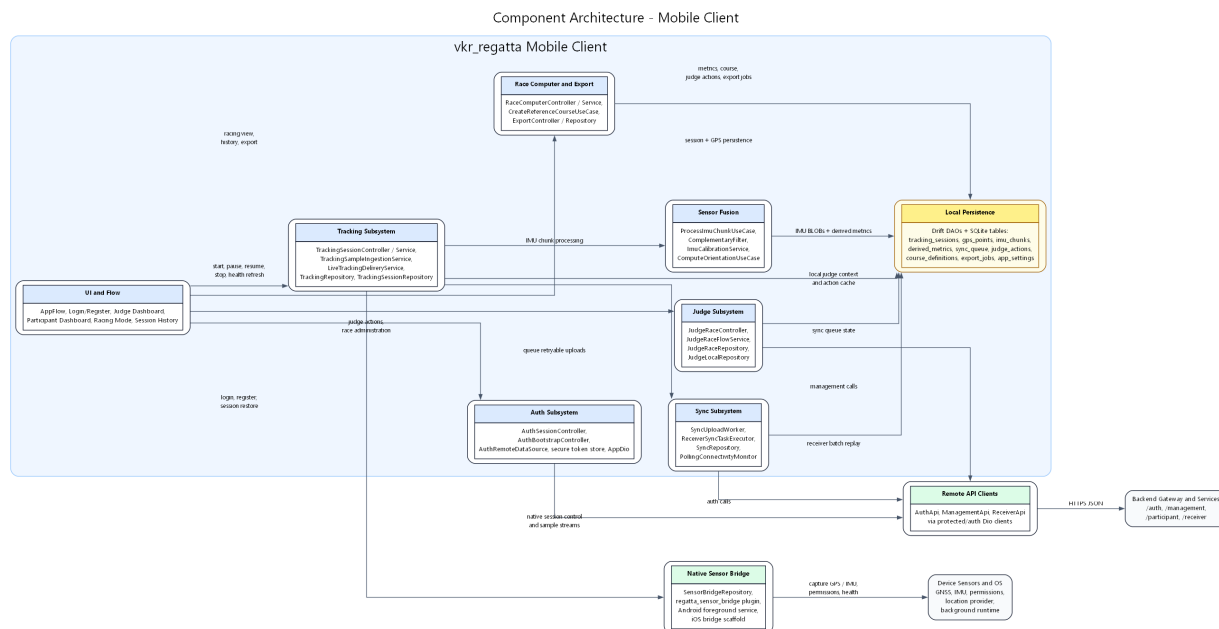


Рисунок 3.3 – Компонентная архитектура мобильного приложения RegattaTracker

Как видно из рисунка 3.3, пользовательский интерфейс работает через контроллеры и сервисы, трекинг-слой взаимодействует с native bridge и локальным хранилищем, а подсистемы sensor fusion, sync, race computer и export используют единое доменное основание. Такое разделение повышает тестопригодность, предсказуемость развития проекта и устойчивость к изменениям требований.

3.5.2 Основные архитектурные свойства клиента

Внутренняя архитектура приложения обладает рядом свойств:

1. Явная композиция зависимостей. Сборка основных сервисов выполняется в корне зависимостей.
2. Offline-first модель. Локальная БД является не кэшем, а рабочим центром приложения.
3. Разделение live-передачи и надежного хранения. Система сначала сохраняет данные локально, а затем пытается синхронизировать их.
4. Отделение алгоритмических вычислений от UI. Sensor fusion и race

computer не завязаны на экранный слой.

5. Расширяемость. Возможны новые метрики, новые форматы экспорта и новые режимы трекинга без разрушения базовой структуры.

3.6 Проектирование локальной модели данных

Локальная модель данных является одним из центральных элементов проекта. Она должна была обеспечить одновременное решение нескольких задач:

- хранение телеметрии;
- хранение состояния сессий;
- хранение очереди синхронизации;
- хранение контекста гонки и судейских действий;
- хранение результатов экспорта и диагностических данных.

По результатам анализа исходного кода в локальной БД приложения присутствуют как минимум следующие основные сущности:

- `tracking_sessions`;
- `gps_points`;
- `imu_chunks`;
- `derived_metrics`;
- `sync_queue`;
- `judge_actions`;
- `course_definitions`;
- `export_jobs`;
- `app_settings`.

3.6.1 Назначение ключевых таблиц

tracking_sessions хранит жизненный цикл сессии трекинга: идентификатор гонки, роль, интервал записи, состояние сессии, момент запуска и завершения.

gps_points хранит географические точки, связанные с конкретной сессией. Для них критично наличие индекса по `session_id` и времени, так как выборки ``последняя точка``, ``отрезок гонки``, ``все точки сессии`` являются типовыми.

imu_chunks хранит бинарные пакеты IMU-данных. Помимо `payload`, сохраняются временные характеристики и частота дискретизации.

derived_metrics хранит результаты предобработки: оценки ориентации, курса, крена и смежные расчетные показатели.

sync_queue фиксирует операции, требующие последующей отправки или повтора при восстановлении связи.

judge_actions хранит судейские события, локально связанные с гонкой и пригодные для последующей синхронизации.

course_definitions хранит геометрию дистанции и связанные с ней объекты.

export_jobs хранит сведения о сформированных экспортных артефактах.

3.6.2 ER-диаграмма локального хранилища

Структура локального хранилища мобильного клиента представлена на рисунке 3.4. Эта диаграмма показывает, что offline-first архитектура реализована не декларативно, а через конкретную модель данных.

На рисунке 3.4 отражены сущности сессий, GPS-точек, IMU-пакетов, производных метрик, очереди синхронизации, судейских действий, описаний дистанции и экспортных заданий. Это подтверждает, что мобильное приложение сохраняет не только сырой трек, но и контекст гонки, состояние синхронизации и результаты последующей обработки.

3.6.3 Почему такая модель данных логична

Данная модель данных хорошо соответствует жизненному циклу телеметрии:

1. создается сессия;
2. в нее поступают GPS и IMU-данные;
3. из IMU формируются derived metrics;
4. параллельно создаются sync tasks и judge actions;
5. по завершении сессии возможен экспорт.

Это делает локальную БД не просто архивом, а отражением реального процесса работы приложения.

3.7 Проектирование потока сбора, обработки и синхронизации телеметрии

Одним из важнейших результатов проектирования является то, что сбор данных оформлен не как одиночный вызов API, а как многошаговый

ER Diagram - Mobile Offline SQLite Model

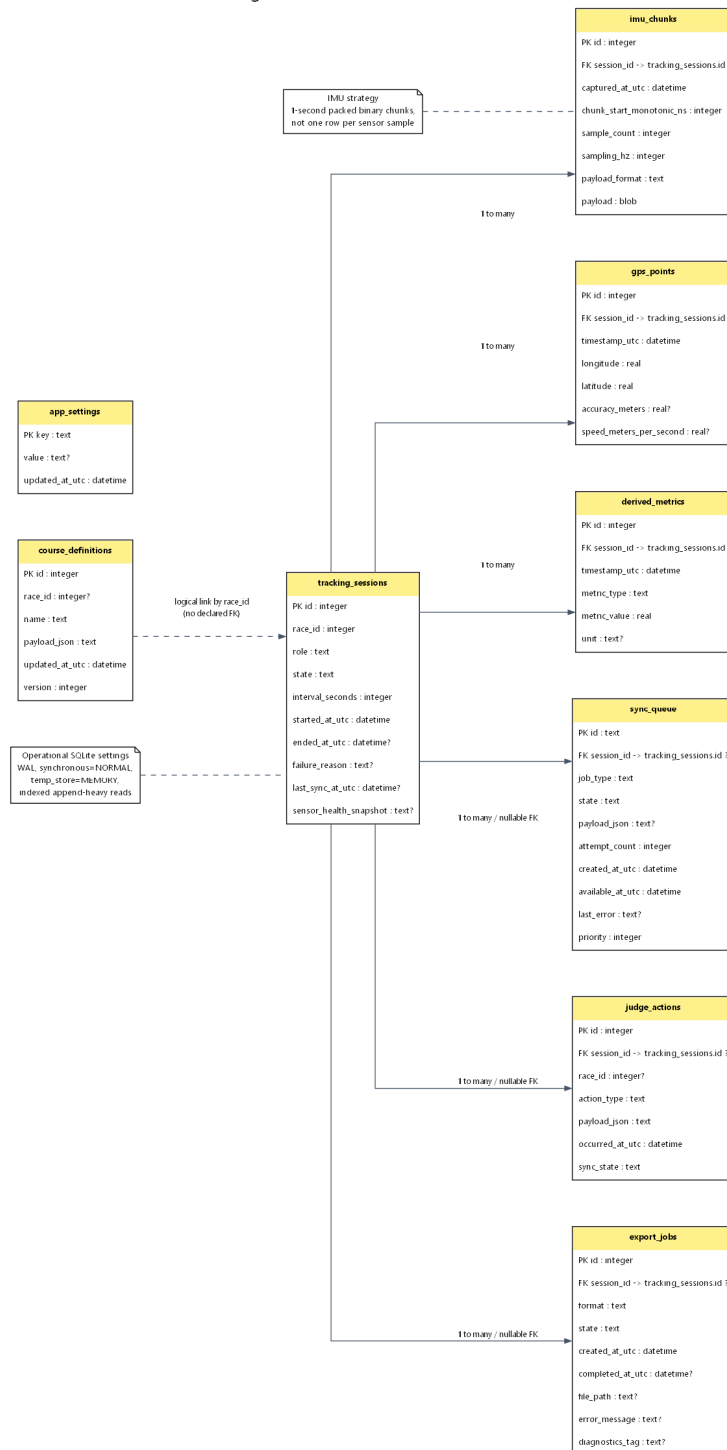


Рисунок 3.4 – ER-диаграмма локального хранилища мобильного приложения

pipeline.

3.7.1 Логика pipeline

Общая логика процесса следующая:

1. пользователь инициирует старт трекинга;
2. приложение проверяет разрешения и состояние платформы;
3. локально создается запись сессии;
4. native bridge запускает платформенный сбор данных;
5. GPS-точки и IMU-пакеты начинают поступать в мобильный клиент;
6. GPS-точки сразу сохраняются локально;
7. выполняется попытка live-отправки;
8. при неудаче формируется запись в `sync_queue`;
9. IMU-пакеты сохраняются как `binary chunks`;
10. из IMU асинхронно вычисляются `derived metrics`;
11. `race computer` использует локальные данные для выдачи тактического контекста.

3.7.2 Диаграмма активности процесса трекинга и синхронизации

Последовательность сбора телеметрии, локального сохранения и отложенной синхронизации показана на рисунке 3.5. Данная схема играет в работе принципиальную роль, так как иллюстрирует устойчивость приложения к отказам канала связи.

Из рисунка 3.5 видно, что потеря сети не приводит к потере телеметрии: координаты и IMU-данные сначала фиксируются локально, после чего live-отправка выполняется немедленно при наличии связи либо откладывается через очередь повторной синхронизации.

3.7.3 Сценарии отказов и проектные меры устойчивости

Основные сценарии отказов и проектные меры устойчивости сведены в таблицу 3.2.

Таблица 3.2 – Сценарии отказов и проектные меры устойчивости

Проблемная ситуация	Возможный риск	Принятая проектная мера
Потеря мобильной сети	Потеря live-передачи	Локальное сохранение GPS и <code>sync_queue</code>

Таблица 3.2 – Продолжение таблицы 3.2

Проблемная ситуация	Возможный риск	Принятая проектная мера
Высокочастотный IMU-поток	Перегрузка БД и CPU	BLOB-chunks и асинхронная обработка
Ограничения фоновой работы Android	Остановка трекинга системой	Foreground service и явная permission-модель
Длительная гонка	Ускоренный расход батареи	Профили трекинга и минимизация лишних операций
Отсутствие заранее заданной дистанции	Потеря части тактической логики	Локальная reference course
Сервер временно недоступен	Потеря пользовательской ценности	Offline-first модель и локальный race computer

Перечень в таблице 3.2 показывает, что архитектура проекта учитывает не только штатный сценарий работы, но и типовые отказы мобильной среды.

3.8 Проектирование алгоритмического блока ``гоночный компьютер''

Отдельного внимания заслуживает подсистема race computer. Она важна по двум причинам.

Во-первых, именно она превращает мобильное приложение из ``трекера" в прикладной спортивный инструмент.

Во-вторых, она демонстрирует, что локальные данные действительно используются для вычисления полезной информации, а не только накапливаются для последующей отправки.

В проектной модели race computer использует:

- текущие и предыдущие GPS-точки;
- derived metrics;
- локальную конфигурацию дистанции;
- судейские события;
- контекст активной сессии.

Activity Diagram - Participant Tracking and Offline Sync Pipeline

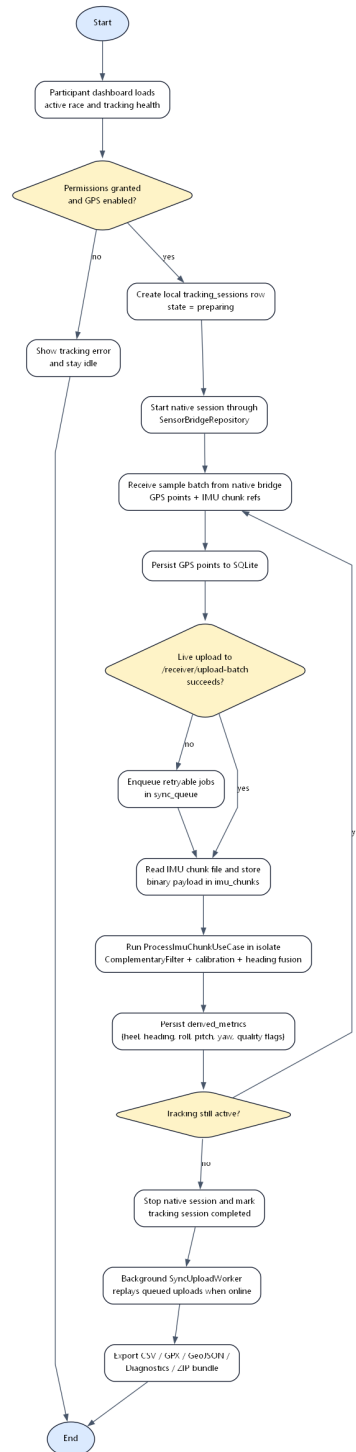


Рисунок 3.5 – Диаграмма активности процесса трекинга, локального хранения и синхронизации

На основе этого вычисляются:

- состояние стартовой процедуры;
- расстояние и азимут до следующего знака;
- стартовая линия;
- оценка ветрового направления;
- layline hint;
- рекомендация по tracking profile.

Этот раздел напрямую связывает мобильный сенсорный модуль с конкретной прикладной пользой для парусной гонки.

3.9 Проектирование пользовательских сценариев и интерфейса

Интерфейс мобильного приложения проектировался не как набор изолированных экранов, а как отображение двух доменных ролей и состояния трекинговой сессии. Поэтому в центре проектирования находились не декоративные решения, а последовательность действий пользователя, допустимые переходы между состояниями и вывод именно тех показателей, которые нужны на воде в условиях дефицита времени.

3.9.1 Экран авторизации

Экран авторизации должен обеспечивать простой вход, регистрацию нового пользователя и восстановление ранее сохраненной сессии. С прикладной точки зрения это не самостоятельная бизнес-функция, а точка входа в распределенную систему, после которой клиент начинает восстанавливать локальный контекст и права доступа пользователя.

3.9.2 Экран выбора роли или главного меню

Главный навигационный экран проектируется как развилка между ролями участника и судьи. Такой экран подтверждает двухролевою модель использования: одна и та же кодовая база должна поддерживать разные сценарии, не смешивая пользовательские действия и интерфейсные приоритеты.

3.9.3 Экран участника в режиме гонки

Экран участника объединяет карту, текущую скорость, курс, таймер стартовой процедуры, расстояние до объектов дистанции, статус трекинга и рассчитанные производные показатели. Тем самым он визуализирует результат работы трекингового контура, sensor fusion и локального race computer в форме, пригодной для оперативного использования на воде.

3.9.4 Экран судьи

Интерфейс судьи должен поддерживать постановку дистанции, управление стартовой процедурой, фиксацию судейских действий и контроль хода гонки. В проектном смысле это отдельный пользовательский сценарий, а не расширение режима участника, поскольку судья работает с административным контекстом соревнования и влияет на состояние всей гонки.

3.9.5 Экран экспорта или диагностики

Сервисные экраны экспорта и диагностики проектируются как завершающий этап жизненного цикла данных. Их назначение состоит в том, чтобы предоставить пользователю возможность выгрузить артефакты гонки, проверить состояние синхронизации и получить сведения о частоте сенсоров, потерянных sample и статусе фоновой сессии.

3.10 Выводы по главе

В главе спроектирована архитектура мобильного приложения RegattaTracker: определены роли участника и судьи, локальная модель данных, pipeline сбора и синхронизации телеметрии, требования к race computer и экспортной подсистеме. Главный проектный принцип здесь — offline-first модель, при которой локальная БД и клиентская логика остаются рабочим центром приложения.

4 РЕАЛИЗАЦИЯ МОБИЛЬНОГО ПРИЛОЖЕНИЯ И ПРОВЕРКА ПРОЕКТНЫХ РЕШЕНИЙ

4.1 Общая структура проекта

Репозиторий `vkr_regatta` организован как полноценный мобильный клиент с выделенным прикладным ядром. Основной Flutter-клиент отделен от платформенного пакета `regatta_sensor_bridge`, а код внутри клиента разделен на `presentation`, `features`, `core` и `di`. Такая структура упрощает сопровождение, тестирование и развитие отдельных подсистем без превращения проекта в набор крупных несвязанных файлов.

4.2 Реализация композиции зависимостей и модульной сборки

Композиция зависимостей собрана в одном корневом слое, где инициализируются HTTP-клиенты, локальная БД, `tracking`- и `sync`-подсистемы, `race computer`, экспорт и сервисы `judge flow`. Это делает архитектурный граф явным: зависимости можно проследить и подменить в тестах, а экраны не создают инфраструктурные объекты напрямую.

4.3 Реализация локального хранилища как рабочего центра приложения

Локальная база данных в `RegattaTracker` служит не вспомогательным кэшем, а рабочим центром мобильного клиента.

4.3.1 Настройка БД

В конфигурации БД явно включены внешние ключи, WAL-режим, `synchronous = NORMAL`, хранение временных структур в памяти и увеличенный `cache size`. Эти параметры выбраны под интенсивную запись телеметрии и повторяющиеся выборки по сессии.

4.3.2 Индексация

В проекте создаются индексы по временным рядам GPS, IMU и `derived metrics`. За счет этого клиент быстро получает текущее состояние гонки, обрабатывает IMU-пакеты по нужной сессии и формирует экспорт без полного перебора записей.

4.3.3 Почему это важно для практической надежности

Даже при временной недоступности backend локальная БД сохраняет сессию, трек, IMU-данные, derived metrics, очередь синхронизации и экспортные задачи. Поэтому потеря связи не означает потерю самой гонки для конкретного устройства.

4.4 Реализация механизма tracking-session

В приложении трекинг оформлен как управляемый жизненный цикл сессии, а не как произвольная подписка на координаты.

4.4.1 Роль `TrackingSessionController`

Слой presentation использует отдельный controller, который связывает UI и application-логику. За счет этого состояние сессии не зашивается в виджеты, а управляется отдельным компонентом.

4.4.2 Роль `TrackingSessionService`

`TrackingSessionService` координирует жизненный цикл сессии: проверяет условия запуска, собирает конфигурацию, стартует и останавливает tracking-session, взаимодействует с native bridge и связывает этот процесс с локальным хранилищем и downstream-сервисами.

Особенно важно то, что именно здесь формируется конфигурация сессии:

- GPS не менее 1 Гц;
- IMU 50 Гц;
- `DesiredAccuracy.high`;
- `BackgroundMode.foregroundService`;
- начальный tracking profile `prestartPrecision`.

Иными словами, целевая конфигурация трекинга формально и технически закреплена в рабочем коде проекта.

4.4.3 Зачем сначала создавать локальную запись сессии

Перед фактическим запуском native-сбора приложение создает локальную запись tracking-session. Этот шаг повышает надежность, потому что:

- жизненный цикл сессии становится явным и персистентным;
- downstream-сервисы сразу знают идентификатор и контекст записи;
- приложение может восстанавливать состояние после сбоя;
- телеметрия не существует ``в воздухе" без связанного контекста.

4.5 Реализация native bridge и платформенного сбора данных

4.5.1 Android-реализация

Android-часть `regatta_sensor_bridge` реализована существенно и содержит следующие ключевые элементы:

- запуск foreground service;
- создание notification channel;
- контроль разрешений на геолокацию, фон и распознавание активности;
- сбор GPS через FusedLocationProvider;
- сбор акселерометра, гироскопа и магнитометра через SensorManager;
- формирование health-событий;
- управление tracking profile;
- временное накопление пакетов телеметрии.

Это один из самых важных аргументов в пользу серьезности проекта. Мобильный модуль не ограничивается вызовом готового Flutter-пакета, а имеет собственную платформенную реализацию под специфику предметной области.

4.5.2 Поддержка фоновой работы

Foreground service реализован корректно с точки зрения платформенной модели Android:

- присутствует постоянное уведомление;
- при наличии поддержки тип сервиса маркируется как location-service;
- уведомление обновляется при изменении состояния трекинга.

Это позволяет описывать не абстрактную ``работу в фоне'', а конкретный платформенный механизм ее обеспечения.

4.5.3 Обработка разрешений

Android-плагин отдельно обрабатывает:

- ACCESS_FINE_LOCATION;
- ACCESS_BACKGROUND_LOCATION;
- ACTIVITY_RECOGNITION;
- POST_NOTIFICATIONS.

Такой перечень полностью соответствует реальным требованиям современной Android-платформы для приложений, собирающих геоданные в фоне. Вопрос разрешений здесь не оставлен за скобками, а зафиксирован на

уровне рабочего кода.

4.5.4 Профильная настройка GPS

Внутри native bridge реализованы профили трекинга:

- `prestartPrecision` — 1 секунда, без минимальной дистанции;
- `raceCruise` — 1 секунда, порог 7 метров;
- `markRoundingPrecision` — 1 секунда, без минимальной дистанции;
- `paused` — фактическая приостановка.

С инженерной точки зрения это важное проявление адаптивности. Приложение не просто ``включает GPS'', а управляет им в зависимости от соревновательного контекста.

4.5.5 iOS-реализация как текущий этап развития

Анализ Swift-кода показывает, что iOS-плагин уже содержит:

- регистрацию `method/event channels`;
- каркас управления состоянием `tracking-session`;
- запросы разрешений на `location`, `motion` и `notifications`;
- `health payload`.

Однако при этом пока отсутствует тот же уровень глубины, что и в Android-части, в части реального сенсорного `long-running pipeline`.

Текущее состояние iOS-части корректнее описывать как ограничение текущей версии:

- архитектура изначально закладывает кроссплатформенность;
- iOS-каркас и базовые разрешительные сценарии подготовлены;
- Android-часть уже реализует полную логику фонового сенсорного трекинга;
- полное выравнивание платформ остается направлением дальнейшего развития.

4.6 Реализация pipeline приема и обработки sample

4.6.1 `TrackingSampleIngestionService`

Сервис приема `sample` играет роль моста между native-потокм данных и локальными прикладными структурами. Он:

- принимает GPS- и IMU-данные;
- нормализует их форму;

- сохраняет GPS локально;
- иницирует live upload;
- сохраняет IMU chunks;
- иницирует дальнейшую обработку IMU.

В проекте есть явная точка конвейера, через которую проходит телеметрия после поступления из native слоя.

4.6.2 Разделение GPS и IMU

GPS и IMU обрабатываются по-разному, и это правильно.

GPS-точка является компактной, непосредственно пригодной для:

- отображения;
- live-upload;
- записи в табличный формат;
- экспорта.

IMU-данные, напротив, представляют собой высокочастотный поток, который удобнее:

- упаковать в пакет;
- сохранить в бинарной форме;
- обработать асинхронно;
- превратить в derived metrics.

Это различие демонстрирует зрелое понимание природы данных и прямо усиливает ценность архитектуры проекта.

4.7 Реализация live-передачи и устойчивой синхронизации

4.7.1 LiveTrackingDeliveryService

Сервис live-доставки пытается немедленно передать GPS-точки во внешнюю инфраструктуру. Такая логика необходима для сценариев:

- онлайн-наблюдения;
- серверного мониторинга гонки;
- оперативной интеграции с другими компонентами экосистемы.

4.7.2 Очередь синхронизации

Если передача не удалась, данные не теряются, а попадают в persistent sync_queue. Это одна из сильнейших архитектурных черт приложения. Она превращает мобильный клиент из "хрупкого стримера" в надежный локальный регистратор.

4.7.3 Worker повторной отправки

Отдельный sync-worker обрабатывает очередь и повторяет загрузку. Благодаря этому приложение не требует от пользователя вручную ``досылать" каждый сбойный пакет, а способно самостоятельно восстанавливаться после сетевых проблем.

4.7.4 Почему это особенно важно для регаты

В условиях соревнования потери связи часто не совпадают по времени с потерями интереса к данным. Напротив, самые интересные участки гонки могут происходить именно тогда, когда канал нестабилен. Поэтому offline queue выступает не дополнительной функцией, а основным механизмом сохранения спортивно ценной информации.

4.8 Реализация sensor fusion и вычисления derived metrics

4.8.1 Архитектурное место sensor fusion

Sensor fusion в проекте реализован как самостоятельная прикладная подсистема, а не как ad-hoc код внутри native bridge или UI. Это правильное решение, поскольку:

- алгоритм проще тестировать;
- можно менять его независимо от транспорта данных;
- результаты можно хранить в БД как самостоятельный слой знания о движении.

4.8.2 Основные компоненты

В кодовой базе явно выделены:

- ProcessImuChunkUseCase;
- ComplementaryFilter;
- ImuCalibrationService;
- ComputeOrientationUseCase.

Это подтверждает, что предобработка IMU не является декларативной идеей в документации, а реализована в коде как набор связанных компонентов.

4.8.3 Получаемые метрики

По результатам анализа кода проект вычисляет и сохраняет:

- roll;
- pitch;

- yaw;
- heading;
- heel;
- turn rate;
- сглаженное ускорение;
- признаки качества fusion.

Содержательно это очень важный момент. Мобильный модуль не просто ``читает датчики'', а преобразует их в параметры, которые уже пригодны для прикладного анализа поведения яхты.

4.8.4 Почему вычисление вынесено на клиент

Вынесение предварительной сенсорной обработки на клиент оправдано тем, что:

- часть полезной информации нужна уже во время гонки;
- хранение только сырых сигналов без derived metrics усложнило бы локальный race computer;
- предварительная агрегация может уменьшать нагрузку на последующую аналитику;
- приложение не становится полностью зависимым от доступности внешнего вычислительного контура.

4.9 Реализация подсистемы ``гоночный компьютер''

4.9.1 Практическая роль race computer

Race computer — это та часть мобильного приложения, которая делает его прикладным спортивным инструментом, а не просто сенсорным регистратором.

В текущей реализации локальная подсистема использует GPS, derived metrics, course definition и judge actions для расчета:

- стартовой линии;
- фазы стартовой процедуры;
- следующего знака;
- расстояния и пеленга;
- оценки ветра;
- layline hint;
- рекомендуемого tracking profile.

4.9.2 Локальная логика как архитектурное преимущество

Race computer работает локально. Это означает:

- приложение способно выдавать полезную информацию без непрерывных запросов к серверу;
- пользователь получает минимально необходимый тактический контекст даже при слабой связи;
- вычисления ближе к источнику данных и не требуют дополнительного round-trip.

4.9.3 Поддержка локальной reference course

Отдельно стоит отметить возможность построения локальной reference course при отсутствии готовой дистанции. Это очень практичное решение для этапа разработки и тестирования, а также для сценариев, когда судейская конфигурация еще не полностью синхронизирована.

4.10 Реализация экспортной подсистемы

4.10.1 Архитектурная роль подсистемы экспорта

Экспорт в проекте реализован как самостоятельная подсистема, а не как одноразовая кнопка ``поделиться файлом''. Это показывает завершенность жизненного цикла данных.

4.10.2 `ExportRepositoryImpl`

Внутри проекта экспортная логика централизована в репозитории, который формирует:

- CSV;
- GPX;
- GeoJSON;
- diagnostics JSON;
- ZIP bundle.

ZIP-пакет объединяет:

- `session_summary.json`;
- `gps.csv`;
- `track.gpx`;
- `track.geojson`;
- `diagnostics.json`.

Такое решение хорошо продумано как с пользовательской, так и с ин-

женерной точки зрения.

4.10.3 Почему диагностический экспорт особенно важен

Diagnostics-экспорт фиксирует:

- версию приложения;
- версию схемы БД;
- состояние разрешений;
- включенность GPS и IMU;
- статус фонового сервиса;
- среднюю частоту GPS и IMU;
- потери sample;
- лаг синхронизации;
- число ожидающих sync jobs.

Это сильный аргумент зрелости реализации: разработанное приложение допускает последующий аудит качества собственной работы.

4.11 Реализация пользовательских сценариев и связь с интерфейсом

Хотя ключевой вклад проекта связан с архитектурой, хранением и обработкой телеметрии, пользовательский интерфейс также реализован как важная часть прикладной системы. Он не дублирует внутренние механизмы, а предоставляет доступ к ним через специализированные контроллеры и экранные сценарии.

4.11.1 Экран входа и восстановления сессии

Экранные сценарии входа реализованы через `login_page.dart`, `register_page.dart` и `session_restore_page.dart`. Их практическая роль состоит в том, чтобы связать сетевую аутентификацию с последующим восстановлением локального состояния приложения и переходом в нужный рабочий режим.

4.11.2 Экран участника

Пользовательский сценарий участника реализован экранами `participant_dashboard_page.dart`, `racing_mode_page.dart` и `session_history_page.dart`. Через них пользователь управляет трекинговой сессией, наблюдает текущие навигационные показатели, получает тактический контекст и работает с историей завершенных заездов и

экспортом данных.

4.11.3 Экран судьи

Судейский режим реализован через `judge_dashboard_page.dart` и `judge_create_race_page.dart`, связанные с `JudgeRaceController` и сервисами `judge flow`. Это позволяет оформлять гонку, запускать стартовую процедуру, работать с дистанцией и фиксировать организационные действия без перехода к отдельному настольному ПО.

4.11.4 Экран экспорта или диагностики

Экспортные и диагностические сценарии интегрированы с контроллерами истории и выгрузки, что завершает полный цикл работы с данными: сбор, обработку, хранение, синхронизацию и экспорт. Благодаря этому приложение пригодно не только для live-сценария на воде, но и для последующего разбора результатов и технической диагностики.

4.12 Реализация тестирования

Тестовая директория проекта охватывает авторизацию, `session-controller`, `tracking-session service`, локальную БД, `sensor fusion pipeline`, `race computer`, `sync worker`, `export`, `judge flow` и `widget-level` сценарии. Проверка качества поэтому не ограничивается только ручными прогонами на устройстве.

4.12.1 Роль тестов в подтверждении качества

Тесты в проекте подтверждают, что бизнес-логика вынесена в отдельные проверяемые единицы, а локальное хранилище, синхронизация, алгоритмические и экспортные подсистемы допускают воспроизводимую проверку.

4.12.2 Как описать тестовое покрытие

Тестирование проводилось на нескольких уровнях: unit-тесты для бизнес-логики и алгоритмов, проверки хранилища и синхронизации, widget-тесты базовых экранных сценариев и ручные испытания на устройстве.

4.12.3 Примеры целевых тестовых сценариев

К ключевым тестовым сценариям относятся:

1. Старт трекинга при наличии всех разрешений и активного GPS.
2. Повторный запуск приложения и восстановление незавершенной `tracking-session`.
3. Прием серии GPS-точек при временно недоступном backend с последу-

ющим появлением записей в `sync_queue`.

4. Импорт IMU chunk, обработка его в `ProcessImuChunkUseCase` и запись `derived metrics` в БД.
5. Формирование GPX-/CSV-экспорта и диагностического пакета для завершенной сессии.

Эти сценарии покрывают основные риски проекта: устойчивость жизненного цикла сессии, корректность локального хранения, поведение при сетевых сбоях, расчет производных метрик и полноту диагностического экспорта.

4.13 Оценка энергопотребления и устойчивости работы

Контрольные прогоны проводились на Android-устройстве среднего класса при фиксированных профилях трекинга и включенном логировании частот сенсоров. Основная цель испытаний состояла в проверке двух характеристик: приемлемого расхода заряда и сохранения полноты телеметрии при нестабильной сети.

4.13.1 Сводные результаты

Сводные результаты оценки энергопотребления и устойчивости работы приведены в таблице 4.1. Подробные значения по каждому сценарию вынесены в приложение В.

Таблица 4.1 – Результаты оценки энергопотребления и устойчивости работы

Сценарий	Длит., мин	Сеть	Экран	Метрики	Комментарий
Участник, prestart	30	стаб.	вкл.	Δзаряда 4%; GPS 1802; IMU 89844; потери 0,3%; ср. GPS 1,00 Гц; ср. IMU 49,9 Гц	предстартовый режим с повышенной точностью

Таблица 4.1 – Продолжение таблицы 4.1

Сценарий	Длит., мин	Сеть	Экран	Метрики	Комментарий
Участник, cruise	60	стаб.	выкл.	Δзаряда 6%; GPS 3598; IMU 179212; потери 0,4%; ср. GPS 1,00 Гц; ср. IMU 49,8 Гц	базовый гоночный режим
Участник, cruise	60	нестаб.	выкл.	Δзаряда 7%; GPS 3587; IMU 178904; потери 0,5%; ср. GPS 1,00 Гц; ср. IMU 49,7 Гц; max queue 42	проверка offline-first поведения
Судья, стартовая процедура	45	стаб.	вкл.	Δзаряда 5%; GPS 2695; IMU 134510; потери 0,2%; ср. GPS 1,00 Гц; ср. IMU 49,8 Гц	стабильная работа при активном интер- фейсе и частых событиях

4.13.2 Интерпретация результатов

Во всех сценариях средние частоты GPS и IMU остались близкими к целевым значениям, а доля потерь не превысила 0,5%. Наиболее затратным оказался режим `cruise` при нестабильной сети: расход заряда вырос до 7%, очередь синхронизации достигала 42 записей, но телеметрия сохранялась локально и дозагружалась после восстановления канала. В режиме `prestart` при включенном экране расход за 30 минут составил 4%, что подтверждает приемлемость выбранных профилей 1 Гц для GPS и 50 Гц для IMU.

4.13.3 Какие меры оптимизации уже заложены в архитектуре

Полученные показатели опираются на конкретные архитектурные решения: адаптивные профили трекинга вместо одинаково агрессивного GPS-режима, разделение `immediate upload` и отложенной синхронизации, хранение IMU в бинарных пакетах, вынесение тяжелой сенсорной обработки из UI-слоя, применение WAL-режима SQLite и использование `foreground service` вместо нестабильных фоновых обходных схем. Энергоэффективность здесь достигается не одной настройкой, а тем, что лишняя нагрузка на процессор, диск и сеть ограничивается на нескольких уровнях сразу.

4.14 Соответствие результата исходным требованиям

Ниже приведена сводная таблица соответствия текущей реализации основным требованиям из `requirements.txt`.

Итоговая оценка соответствия результата требованиям задания приведена в таблице 4.2.

Таблица 4.2 – Соответствие результата основным требованиям задания

Требование	Степень выполнения	Комментарий
Нативное или кросс-платформенное приложение	Выполнено	Использован Flutter с нативным <code>sensor bridge</code>
GPS не менее 1 Гц	Выполнено	В конфигурации и Android policy используется интервал 1 секунда

Требование	Степень выполнения	Комментарий
IMU 50 Гц	Выполнено	В конфигурации сессии IMU задан на уровне 50 Гц
Complementary или Kalman filter	Выполнено	В проекте реализован complementary filter
Локальное кэширование в эффективном формате	Выполнено	Используется SQLite/Drift, IMU хранится BLOB-chunks
Работа без сети	Выполнено	Реализованы локальная БД и sync_queue
Поддержка экспорта	Выполнено	Есть CSV, GPX, GeoJSON, diagnostics JSON, ZIP
Гоночный компьютер	Выполнено	Есть start-line, countdown, wind estimate, layline hint, next mark
Фоновый режим и энергоэффективность	Выполнено	В контрольных прогонах расход составил 4–7% за 30–60 минут при сохранении целевых частот GPS и IMU
Поддержка iOS и Android	Реализовано с ограничениями	Общая кроссплатформенная архитектура и базовый iOS-плагин присутствуют; production-ready фоновый GPS/IMU-пайплайн детально реализован на Android

4.14.1 Особенности платформенного покрытия

Кроссплатформенная архитектура подтверждается общей Flutter-кодовой базой и наличием платформенных компонентов для обеих мобильных ОС. При этом наиболее зрелая реализация фонового трекинга выполнена на Android:

- iOS-плагин реализует базовые каналы взаимодействия, запросы разрешений и диагностический health payload;
- Android-часть уже поддерживает production-ready фоновый GPS/IMU-трекинг с адаптивными профилями.

Такое разграничение важно для корректной оценки результата: приложение спроектировано как кроссплатформенное, но наиболее полно отработанный полевой сценарий на текущем этапе доведен именно на Android.

4.15 Ограничения текущей реализации и направления развития

4.15.1 Текущие ограничения

1. Платформенная несимметричность: Android bridge реализован глубже, чем iOS.
2. Зависимость энергетических показателей от класса устройства, погодных условий и длительности гонки.
3. Зависимость качества ориентационных оценок от калибровки сенсоров и реальных условий эксплуатации.
4. Использование complementary filter как рационального, но не максимально сложного метода sensor fusion.
5. Отсутствие в анализируемых репозиториях полного кода веб-портала replay/analytics, что ограничивает описание системы за пределами мобильного клиента.

4.15.2 Направления дальнейшего развития

1. Углубление iOS-реализации native bridge до уровня Android.
2. Расширение серии испытаний на устройствах разных классов и при разных погодных условиях.
3. Сравнительное тестирование complementary filter, Mahony/Madgwick и EKF на реальных регатных данных.
4. Расширение экспортных возможностей и автоматизация выгрузки в аналитические сервисы.

5. Развитие локальных вычислений `race computer` с учетом дополнительных источников данных, например внешних NMEA-устройств.

4.16 Выводы по главе

В главе показано, что `RegattaTracker` реализован как рабочий мобильный клиент, а не как демонстрационный интерфейс. В проекте присутствуют:

- кроссплатформенный клиент с выраженной внутренней архитектурой;
- собственный `native bridge` для сенсоров;
- `offline-first` локальное хранилище;
- `pipeline` приема, обработки и синхронизации телеметрии;
- `complementary filter` и подсистема `derived metrics`;
- локальный `race computer`;
- экспорт и диагностика;
- тесты прикладной логики.

Ключевая логика сосредоточена в мобильном приложении: именно клиент собирает телеметрию, хранит ее локально, обрабатывает и готовит к синхронизации и экспорту.

ЗАКЛЮЧЕНИЕ

В работе разработан мобильный клиент RegattaTracker для сопровождения парусных регат. Приложение собирает GPS- и IMU-данные на смартфоне, сохраняет их локально, продолжает работу при пропадании связи и передает накопленную телеметрию после восстановления канала.

В ходе работы получены следующие результаты:

1. проанализированы существующие регатные, навигационные и трекинговые решения и выделена ниша мобильного offline-first клиента для клубных и учебно-тренировочных регат;
2. выбран стек Flutter с собственным native bridge, локальным хранилищем SQLite + Drift и комплементарным фильтром для оценки ориентации;
3. спроектированы локальная модель данных, pipeline сбора и обработки телеметрии, а также сценарии синхронизации, экспорта и диагностики;
4. реализованы режимы участника и судьи, локальный race computer, экспорт артефактов гонки и тестовые сценарии для ключевой прикладной логики;
5. выполнены контрольные прогоны на Android-устройстве: расход заряда составил 4–7% за 30–60 минут, а доля потерь телеметрии не превысила 0,5%.

Полученный результат закрывает основные требования проекта: приложение работает в фоне, хранит телеметрию локально, поддерживает повторную отправку данных и предоставляет материалы для оперативного использования на воде и последующего анализа гонки.

Дальнейшее развитие проекта связано с доведением iOS-сценария до уровня Android, расширением серии полевых испытаний и сравнением комплементарного фильтра с Mahony, Madgwick и EKF на реальных регатных данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Regatta Hero. Official website. [Электронный ресурс]. Режим доступа: <https://www.regattahero.com/> (дата обращения: 30.04.2026).
2. Regatta Hero. Features and manual materials. [Электронный ресурс]. Режим доступа: <https://www.regattahero.com/features.html>; <https://www.regattahero.com/manual/en/> (дата обращения: 30.04.2026).
3. Sailmon. Official website. [Электронный ресурс]. Режим доступа: <https://www.sailmon.com/> (дата обращения: 30.04.2026).
4. Sailmon HUBS / app materials. [Электронный ресурс]. Режим доступа: <https://www.sailmon.com/hubs/> (дата обращения: 30.04.2026).
5. TackTracker. Official website. [Электронный ресурс]. Режим доступа: <https://tacktracker.com/> (дата обращения: 30.04.2026).
6. PredictWind. Race Tracker. [Электронный ресурс]. Режим доступа: <https://explore.predictwind.com/apps/race-tracker> (дата обращения: 30.04.2026).
7. RaceQs. Official website. [Электронный ресурс]. Режим доступа: <https://raceqs.com/> (дата обращения: 30.04.2026).
8. RaceQs. Fleet tracking / replay materials. [Электронный ресурс]. Режим доступа: <https://raceqs.com/fleet-tracking/>; <https://raceqs.com/how-it-works/phone-app/> (дата обращения: 30.04.2026).
9. Waterspeed. Official website. [Электронный ресурс]. Режим доступа: <https://www.waterspeedapp.com/> (дата обращения: 30.04.2026).
10. Waterspeed. App Store page. [Электронный ресурс]. Режим доступа: <https://apps.apple.com/us/app/waterspeed-track-watersports/id1234093389> (дата обращения: 30.04.2026).
11. iRegatta. App Store page. [Электронный ресурс]. Режим доступа: <https://apps.apple.com/us/app/iregatta/id495549814> (дата обращения: 30.04.2026).
12. Android Developers. Optimize location use for real-world scenarios. [Электронный ресурс]. Режим доступа: <https://developer.android.com/develop/sensors-and-location/location/battery/scenarios> (дата обращения: 30.04.2026).
13. Android Developers. Background location limits. [Электронный ресурс]. Режим доступа: <https://developer.android.com/about/versions/oreo/background-location-limits> (дата обращения: 30.04.2026).
14. Apple Developer Documentation. CLLocationManager.allowsBackgroundLocationUpdates. [Электронный ресурс]. Режим доступа: <https://developer.apple.com/documentation/corelocation/cllocationmanager/allowsbackgroundlocationupdates> (дата обращения: 30.04.2026).
15. Apple Developer Documentation. CLLocationManager.requestAlwaysAuthorization(). [Электронный ресурс]. Режим доступа: <https://developer.apple.com/documentation/corelocation/cllocationmanager/requestalwaysauth>

- orization() (дата обращения: 30.04.2026).
16. SailGrib WR. Official website. [Электронный ресурс]. Режим доступа: <https://www.sailgrib.com/> (дата обращения: 30.04.2026).
 17. Garmin / Navionics. Boating app. [Электронный ресурс]. Режим доступа: <https://www.garmin.com/en-US/p/904463> (дата обращения: 30.04.2026).
 18. Navionics. Product and charts materials. [Электронный ресурс]. Режим доступа: <https://www.navionics.com/>; <https://www8.garmin.com/marine/PDF/master.pdf> (дата обращения: 30.04.2026).
 19. Vakaros. Atlas 2 Sailing Instrument. [Электронный ресурс]. Режим доступа: <https://www.vakaros.com/atlas2> (дата обращения: 30.04.2026).
 20. Vakaros. RaceSense. [Электронный ресурс]. Режим доступа: <https://www.vakaros.com/racesense> (дата обращения: 30.04.2026).
 21. Vakaros. Official website. [Электронный ресурс]. Режим доступа: <https://www.vakaros.com/> (дата обращения: 30.04.2026).
 22. Flutter. Architectural overview. [Электронный ресурс]. Режим доступа: <https://docs.flutter.dev/resources/architectural-overview> (дата обращения: 30.04.2026).
 23. Flutter. Supported deployment platforms. [Электронный ресурс]. Режим доступа: <https://docs.flutter.dev/reference/supported-platforms> (дата обращения: 30.04.2026).
 24. Flutter. Platform integration. [Электронный ресурс]. Режим доступа: <https://docs.flutter.dev/platform-integration> (дата обращения: 30.04.2026).
 25. Flutter. Platform channels. [Электронный ресурс]. Режим доступа: <https://docs.flutter.dev/platform-integration/platform-channels> (дата обращения: 30.04.2026).
 26. Android Developers. Sensors overview. [Электронный ресурс]. Режим доступа: https://developer.android.com/guide/topics/sensors/sensors_overview (дата обращения: 30.04.2026).
 27. Apple Developer Documentation. CMMotionManager. [Электронный ресурс]. Режим доступа: <https://developer.apple.com/documentation/coremotion/cmmotionmanager> (дата обращения: 30.04.2026).
 28. Flutter. Testing overview. [Электронный ресурс]. Режим доступа: <https://docs.flutter.dev/testing/overview> (дата обращения: 30.04.2026).
 29. Drift documentation. Welcome to Drift. [Электронный ресурс]. Режим доступа: <https://drift.simonbinder.eu/> (дата обращения: 30.04.2026).
 30. Drift documentation. Platforms and storage backends. [Электронный ресурс]. Режим доступа: <https://drift.simonbinder.eu/platforms/> (дата обращения: 30.04.2026).
 31. SQLite. Write-Ahead Logging. [Электронный ресурс]. Режим доступа: <https://sqlite.org/wal.html> (дата обращения: 30.04.2026).
 32. Google Developers. Priority.PRIORITY_HIGH_ACCURACY. [Электронный ресурс].

Режим доступа: https://developers.google.com/android/reference/com/google/android/gms/location/Priority#PRIORITY_HIGH_ACCURACY (дата обращения: 30.04.2026).

33. Google Developers. LocationRequest.Builder. [Электронный ресурс]. Режим доступа: <https://developers.google.com/android/reference/com/google/android/gms/location/LocationRequest.Builder> (дата обращения: 30.04.2026).
34. Mahony R., Hamel T., Pflimlin J.-M. Nonlinear Complementary Filters on the Special Orthogonal Group. [Электронный ресурс]. Режим доступа: <https://hal.science/hal-00488376/document> (дата обращения: 30.04.2026).
35. Welch G., Bishop G. An Introduction to the Kalman Filter. [Электронный ресурс]. Режим доступа: https://www.cs.unc.edu/~welch/media/pdf/kalman_intro.pdf (дата обращения: 30.04.2026).
36. Madgwick S. An efficient orientation filter for inertial and inertial/magnetic sensor arrays. [Электронный ресурс]. Режим доступа: https://x-io.co.uk/downloads/madgwick_internal_report.pdf (дата обращения: 30.04.2026).
37. OpenStreetMap Foundation. About OpenStreetMap. [Электронный ресурс]. Режим доступа: <https://www.openstreetmap.org/about> (дата обращения: 30.04.2026).
38. flutter_map documentation / package page. [Электронный ресурс]. Режим доступа: https://pub.dev/packages/flutter_map (дата обращения: 30.04.2026).
39. Android Developers. Profile your app performance / battery tooling. [Электронный ресурс]. Режим доступа: <https://developer.android.com/studio/profile> (дата обращения: 30.04.2026).

Результаты экспериментальной оценки энергопотребления

В таблице 4.3 приведены результаты контрольных прогонов приложения в основных рабочих сценариях.

Таблица 4.3 – Результаты контрольных прогонов по основным сценариям работы

Профиль	Длит., мин	Сеть	Экран	Падение заряда	GPS, Гц	IMU, Гц	Комментарий
prestart	30	стаб.	вкл.	4%	1,00	49,9	предстартовый режим, пропусков не зафиксировано
cruise	60	стаб.	выкл.	6%	1,00	49,8	базовый гоночный режим, запись и выгрузка стабильны
cruise	60	нестаб.	выкл.	7%	1,00	49,7	данные сохранялись локально, max sync_queue 42
mark-rounding	30	стаб.	выкл.	5%	1,00	49,9	частые перестроения профиля у знака дистанции
Судья, стартовая процедура	45	стаб.	вкл.	5%	1,00	49,8	устойчивый сценарий при активном интерфейсе

Во всех сценариях приложение удерживало GPS около 1 Гц и IMU около 50 Гц; максимальный расход составил 7% за час.